

机器学习导论-无监督学习

Introduction to Machine Learning-**Unsupervised Learning**

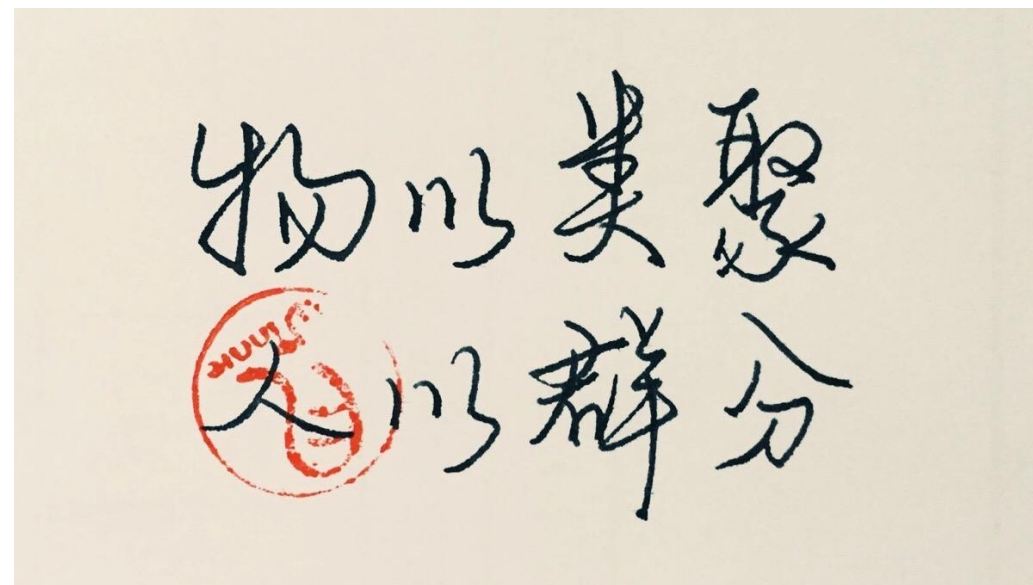
李文斌

<https://cs.nju.edu.cn/liwenbin>

liwenbin@nju.edu.cn

2024年05月07日

引入：聚类算法



大纲

□ **相关概念**

□ **距离度量**

□ **聚类准则**

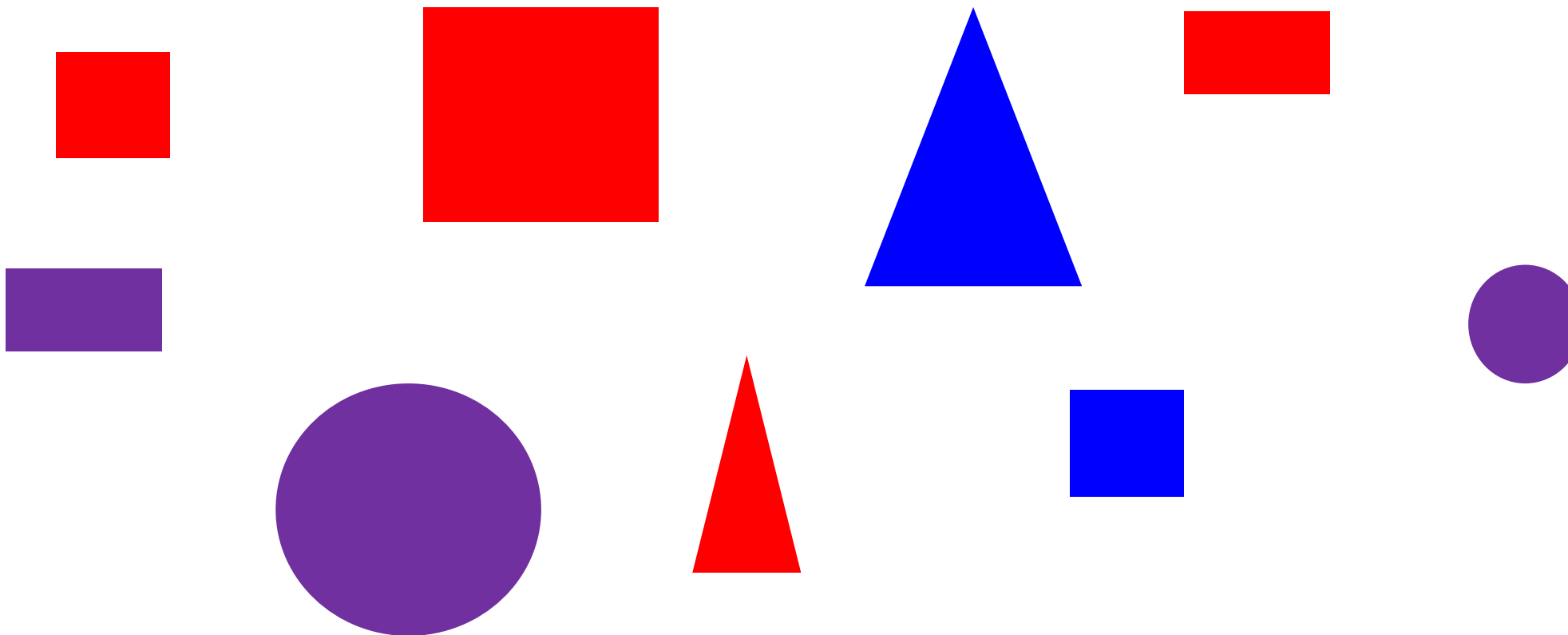
□ **聚类方法**

□ **聚类评价**

□ **前沿进展**

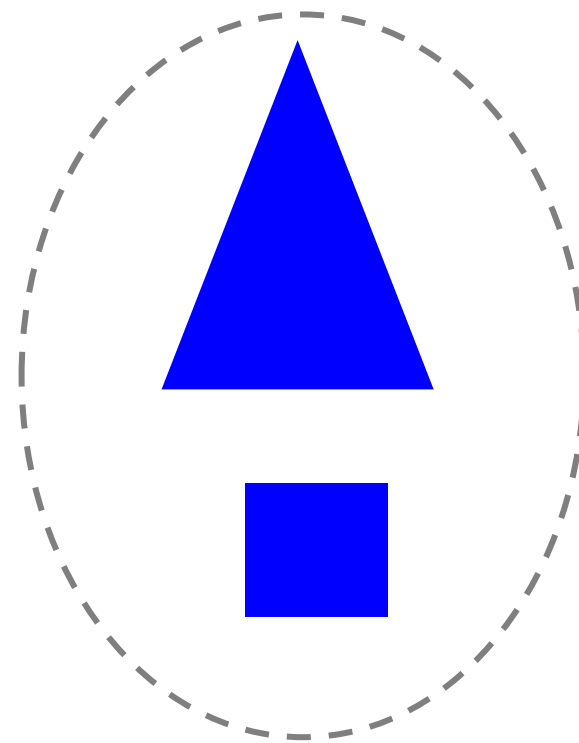
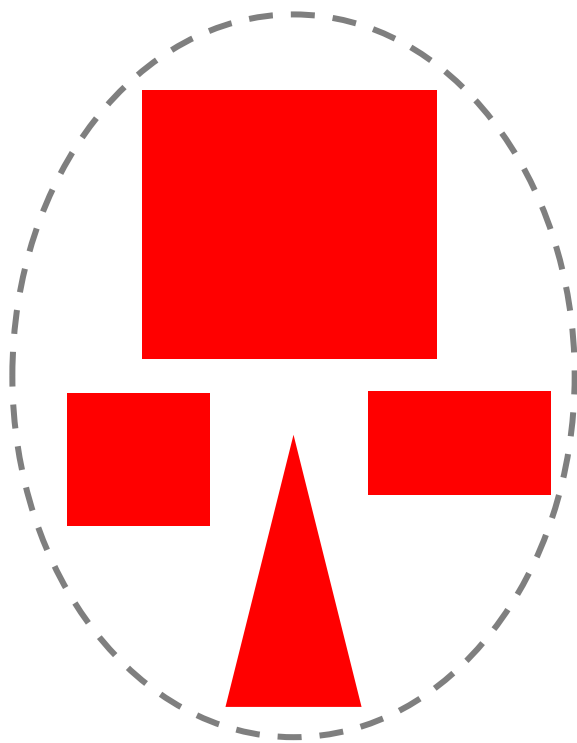
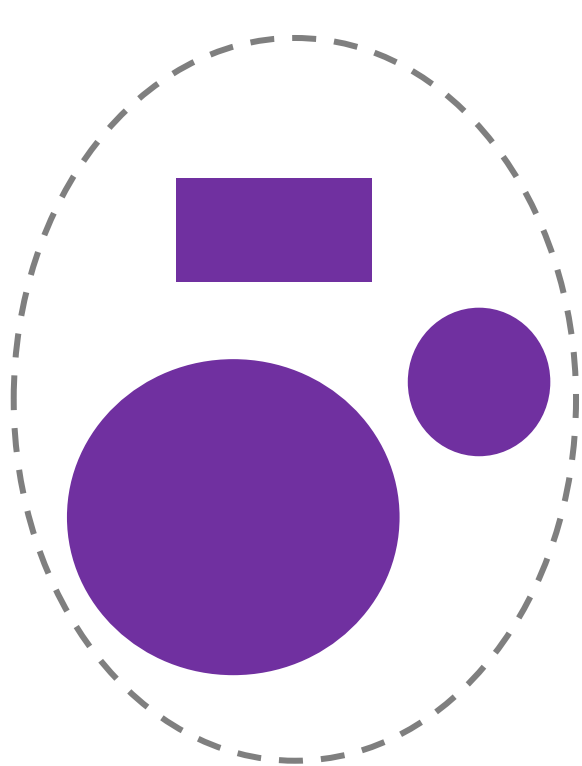
引入：聚类算法

□ 对下面图形进行划分 (幼儿园作业)



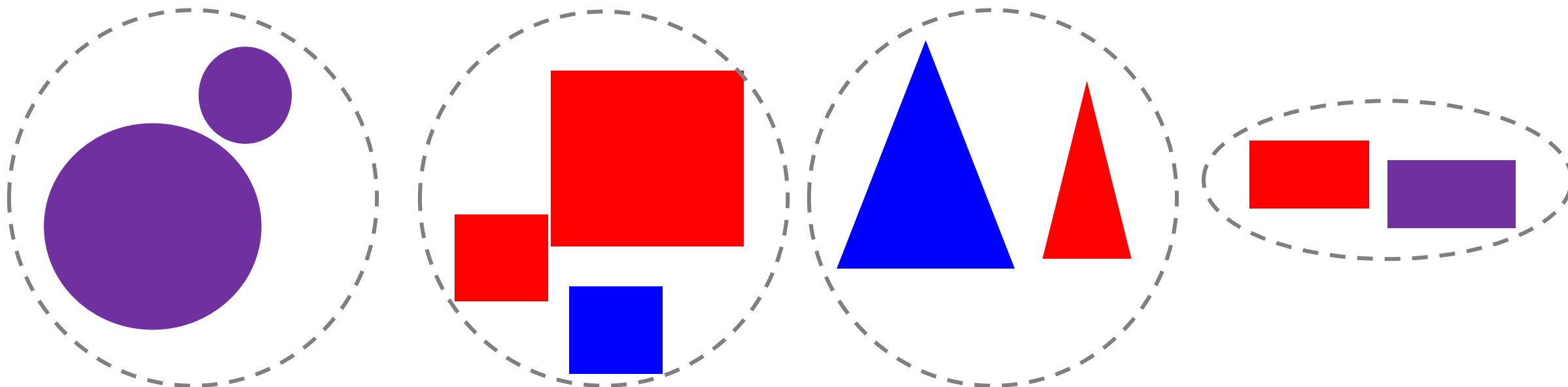
引入：聚类算法

□ 颜色



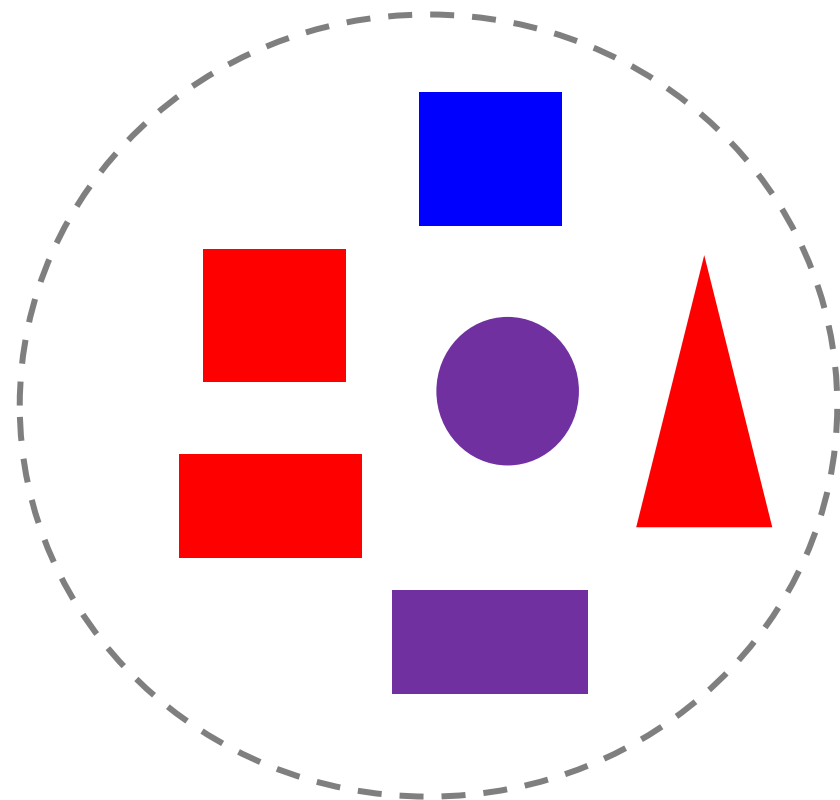
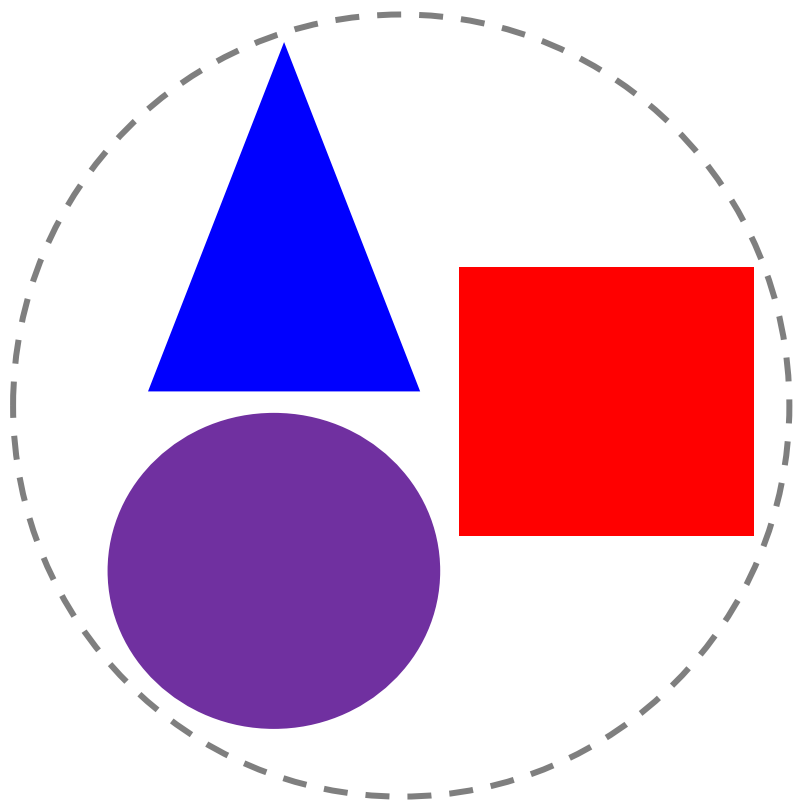
引入：聚类算法

□ 形状



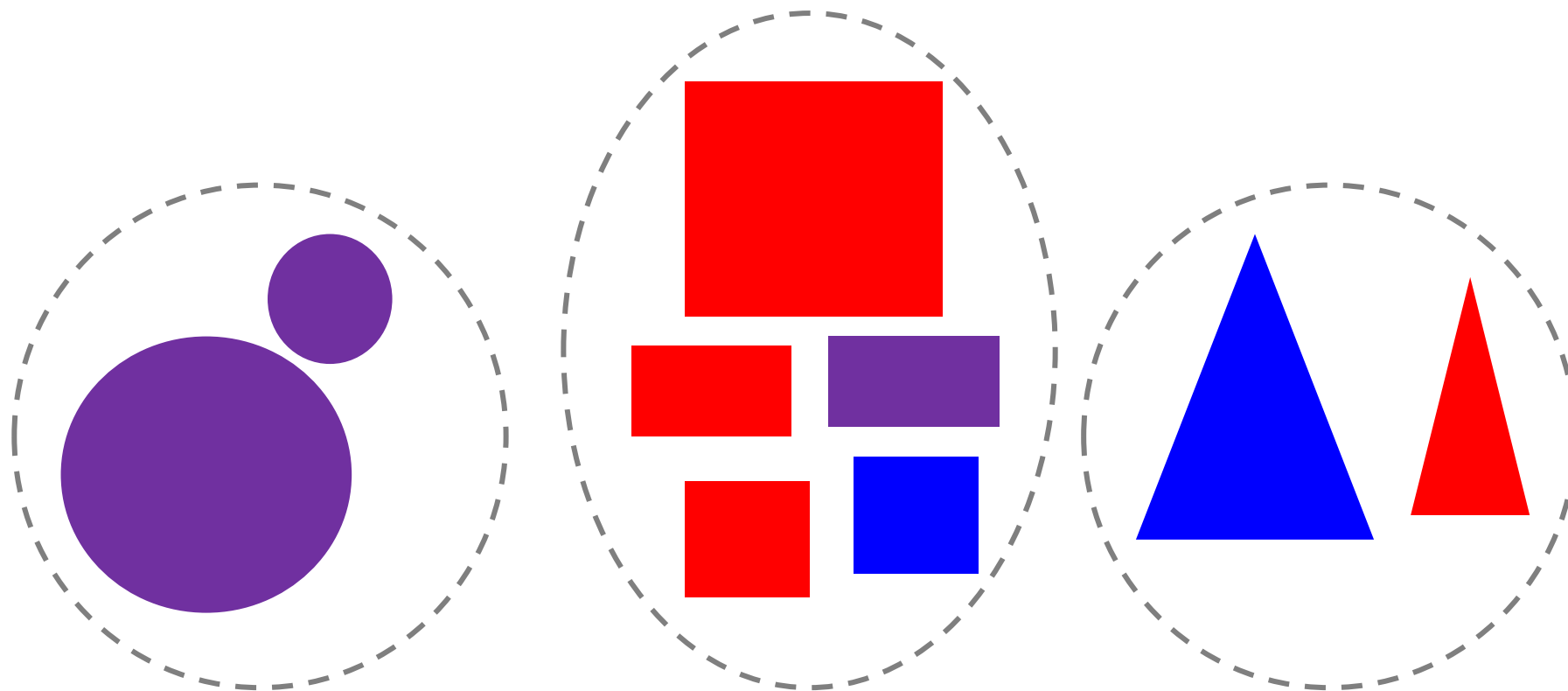
引入：聚类算法

□ 大小



引入：聚类算法

□ 顶点数



引入：聚类算法

聚类的“好坏”不存在绝对标准！

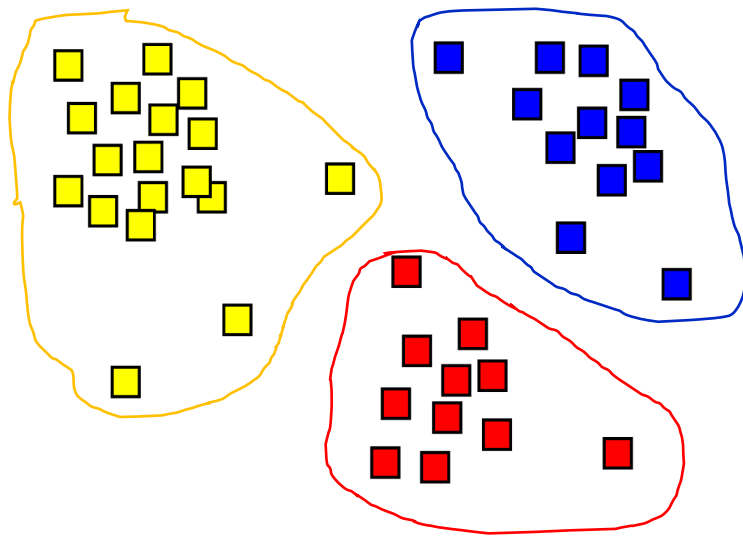
**The goodness of clustering depends on the
opinion of the user**

聚类也许是机器学习中“新算法”出现最多、最快的领域，
总能找到一个新的“标准”，使以往算法对它无能为力

相关概念

□ 聚类（簇、类）：**数据对象的集合**

- 在同一个簇/类中，数据对象是相似的
- 不同簇/类之间的数据对象是不相似的



相关概念

□ 聚类算法

- 根据给定的**相似性评价标准**，将一个数据集合分组/划分成几个聚类（簇）

□ 数学形式化

- 样本集合： $D = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m\}, \mathbf{x}_i \in \mathbb{R}^d$
- 聚类成 k 个簇
 - $\{C_l | l = 1, 2, \dots, k\}$
 - $C_{l'} \cap_{l' \neq l} C_l = \emptyset$
 - $D = \bigcup_{l=1}^k C_l$

相关概念

□ 聚类的依据

- 将整个数据集中每个样本的特征向量看成是分布在特征空间中的一些点，
点与点之间的距离即可作为相似性度量依据
- 聚类分析是根据不同样本之间的差异，根据距离函数的规律（大小）进行聚类的

相关概念

□ 一个好的聚类算法

- 聚类（簇）内部高相似性
- 聚类（簇）之间低相似性



相关概念

□ 聚类算法中“类”的特征：

- 聚类所说的类不是事先给定的，而是根据数据的相似性和距离来划分
(**无监督的算法**)
- 聚类的数目和结构都没有事先假定

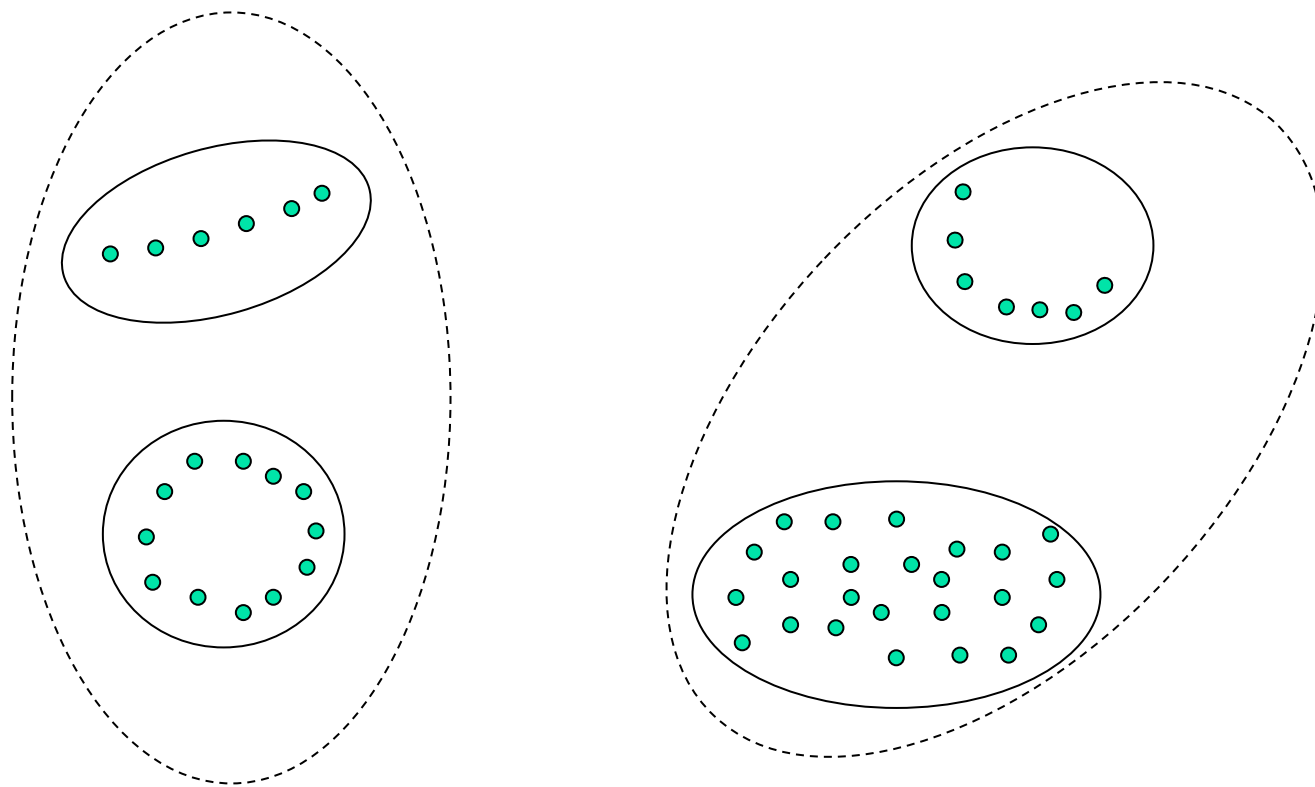
□ 聚类方法的目的是寻找数据中：

- 潜在的**自然分组结构**
- 感兴趣的**关系**



相关概念

□ 粒度对聚类的影响



数据的粗聚类是2类,细聚类为4类

相关概念

□ 聚类的关键

- 特征的选取或设计
- 距离度量函数的选择

□ 聚类分析的有效性：

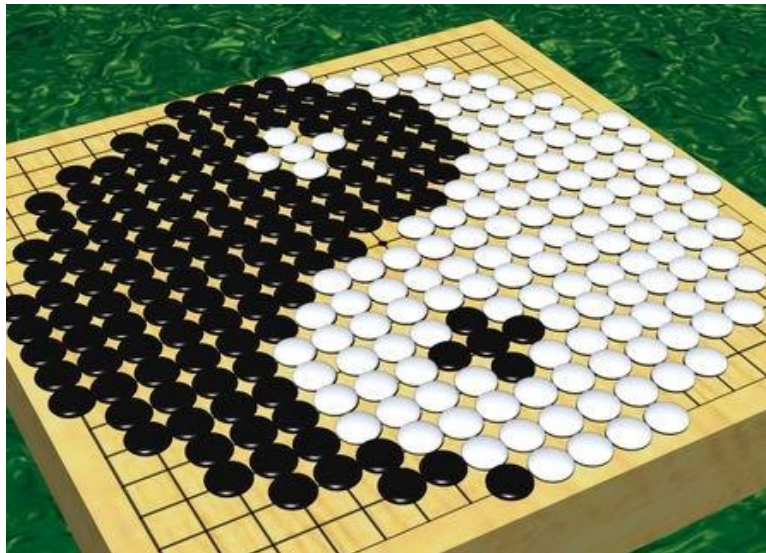
- 若数据点的分布是一群一群的，同一群样本密集（距离很近），不同群样本距离很远，则很容易聚类；
- 若样本集的分布聚成一团，不同群的样本混在一起，则很难分类；
- 对具体数据做聚类分析的**关键是选取合适的特征**。特征选取得好，容易区分，选取得不好，很难区分。

聚类分析方法是否有效，与数据分布形式有很大关系！

相关概念

□ 两类聚类实例：一摊黑白围棋子

- 选**颜色**作为特征进行聚类，用“1”代表白，“0”代表黑，则很容易分类；
- 选**大小**作为特征进行聚类，则白子和黑子的特征相同，不能分类（把白子和黑子分开）



大纲

- 相关概念
- 距离度量
- 聚类准则
- 聚类方法
- 聚类评价
- 前沿进展

距离度量

□ 目的：度量同类样本间的相似性和不同类样本间的差异性

□ 度量函数和度量空间

- 度量空间是一个有序对，记作 (X, d) ，其中 X 是一个集合， d 是 X 上的度量函数：它把 X 中的每一对点 x, y 映射到一个非负实数，并满足以下四条公理：
- 非负性： $d(x, y) \geq 0$
- 唯一性： $d(x, y) = 0 \leftrightarrow x = y$
- 对称性： $d(x, y) = d(y, x)$
- 三角不等式： $x, y, z \in X, d(x, z) \leq d(x, y) + d(y, z)$

距离度量

□ 常用度量函数 ($\mathbf{x}_i, \mathbf{x}_j \in \mathbb{R}^n$)

- 欧氏距离: $d(\mathbf{x}_i, \mathbf{x}_j) = \sqrt{(\mathbf{x}_i - \mathbf{x}_j)^T (\mathbf{x}_i - \mathbf{x}_j)}$
- 余弦相似性: $s(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{x}_i^T \mathbf{x}_j}{\|\mathbf{x}_i\| \|\mathbf{x}_j\|}$
- 曼哈顿距离: $d(\mathbf{x}_i, \mathbf{x}_j) = \|\mathbf{x}_i - \mathbf{x}_j\|_1$
- 切比雪夫距离: $d(\mathbf{x}_i, \mathbf{x}_j) = \|\mathbf{x}_i - \mathbf{x}_j\|_\infty$
- 马氏距离: $d_M(\mathbf{x}_i, \mathbf{x}_j) = \sqrt{(\mathbf{x}_i - \mathbf{x}_j)^T \mathbf{M} (\mathbf{x}_i - \mathbf{x}_j)}$

距离度量

□ 常用度量函数 ($\mathbf{x}_i, \mathbf{x}_j \in \mathbb{R}^n$)

- 欧式距离: $d(\mathbf{x}_i, \mathbf{x}_j) = \|\mathbf{x}_i - \mathbf{x}_j\|_2$
- 曼哈顿距离: $d(\mathbf{x}_i, \mathbf{x}_j) = \|\mathbf{x}_i - \mathbf{x}_j\|_1$
- 切比雪夫距离: $d(\mathbf{x}_i, \mathbf{x}_j) = \|\mathbf{x}_i - \mathbf{x}_j\|_\infty$



闵可夫斯基距离
Minkowski distance

$$d(\mathbf{x}_i, \mathbf{x}_j) = \left(\sum_{u=1}^n |x_{iu} - x_{ju}|^p \right)^{\frac{1}{p}}$$

聚类准则

□ 类的定义

- 类的定义有很多种，类的划分具有人为规定性。一个聚类结果的优劣最后只能根据实际来评价。
- **定义之一：** 设集合 S 中任意元素 x_i 与 x_j 间的距离有：

$$d(x_i, x_j) \leq h$$

其中 h 为给定的阈值，称 S 对于阈值 h 组成一类。

聚类准则

□ 类的定义

- 聚类有了样本的相似性度量/距离度量，还需要一种基于数值的聚类准则，才能将相似的样本分在同一类，相异的样本分在不同的类
- 判别聚类结果好坏的一般标准：类内距离小，类间距离大或者，
簇内相似度（intra-cluster similarity）高，
簇间相似低（inter-cluster similarity）低。
- 需要一个能对聚类过程或聚类结果的优劣进行评估的准则函数。如果聚类准则函数选择得好，聚类质量就会高。

大纲

- 相关概念
- 距离度量
- 聚类准则
- 聚类方法
- 聚类评价
- 前沿进展

聚类准则

□ 试探方法

- 凭直观感觉或经验，针对实际问题定义一种距离度量的**阈值**，然后按**最近邻规则**指定某些样本属于某一个聚类类别。
- 例如对欧氏距离，它反映了样本间的近邻性，但将一个样本分到不同类别中的哪一个时，还必须规定一个距离度量的阈值作为聚类的判别准则。

聚类准则

□ 聚类准则函数方法

- **依据：**由于聚类是将样本进行分类以使类别之间的分离性尽可能大，因此聚类准则应是反映类别间相似性或分离性的函数；
- 每个类别都是由一系列样本组成的，因此一般来说类别（sample sets）的可分离性和样本（samples）的可分离性是直接相关的；
- 可以定义聚类准则函数为样本集 $\{x\}$ 和类别 $\{S_j, j = 1, 2, \dots, c\}$ 的函数，从而使聚类分析转化为寻找准则函数极值的最优化问题。

聚类准则

□ 聚类准则函数方法

- 一种聚类准则函数 J 的定义

$$J = \sum_{j=1}^c \sum_{\mathbf{x} \in S_j} \|\mathbf{x} - \mathbf{m}_j\|^2$$

- J 代表了属于 c 个聚类类别的全部样本与其相应类别均值 m_j 之间的误差平方和
- 对于不同的聚类形式, J 值是不同的
- 目的: 求取使 J 值达到最小的聚类形式

大纲

- 相关概念
- 距离度量
- 聚类准则
- **聚类方法**
- 聚类评价
- 前沿进展

聚类方法

- **基于试探的聚类搜索算法**
- **系统聚类法**
- **动态聚类法**

基于试探的聚类搜索算法

□ 按最近邻规则的简单试探法

- 给定 N 个待分类的数据样本 $\{x_1, x_2, \dots, x_N\}$, 要求按距离阈值 T , 将它们分类到聚类中心 $z_1, z_2, \dots$
- **第一步:**
 - 任取一样本 x_i 作为一个聚类中心的初始值, 例如令 $z_1 = x_1$
 - 计算 $D_{21} = \|x_2 - z_1\|_2$
 - 若 $D_{21} > T$, 则确定一个新的聚类中心 $z_2 = x_2$
 - 否则 x_2 属于以 z_1 为中心的聚类
- **第二步:**
 - 假设已经有聚类中心 z_1, z_2
 - 计算 $D_{31} = \|x_3 - z_1\|_2$, $D_{32} = \|x_3 - z_2\|_2$
 - 若 $D_{31} > T$ 且 $D_{32} > T$, 则得一个新的聚类中心 $z_3 = x_3$
 - 否则 x_3 属于离 z_1 和 z_2 中的最近者
 -
 - 如此重复下去, 直至将 N 个样本分类完毕。

基于试探的聚类搜索算法

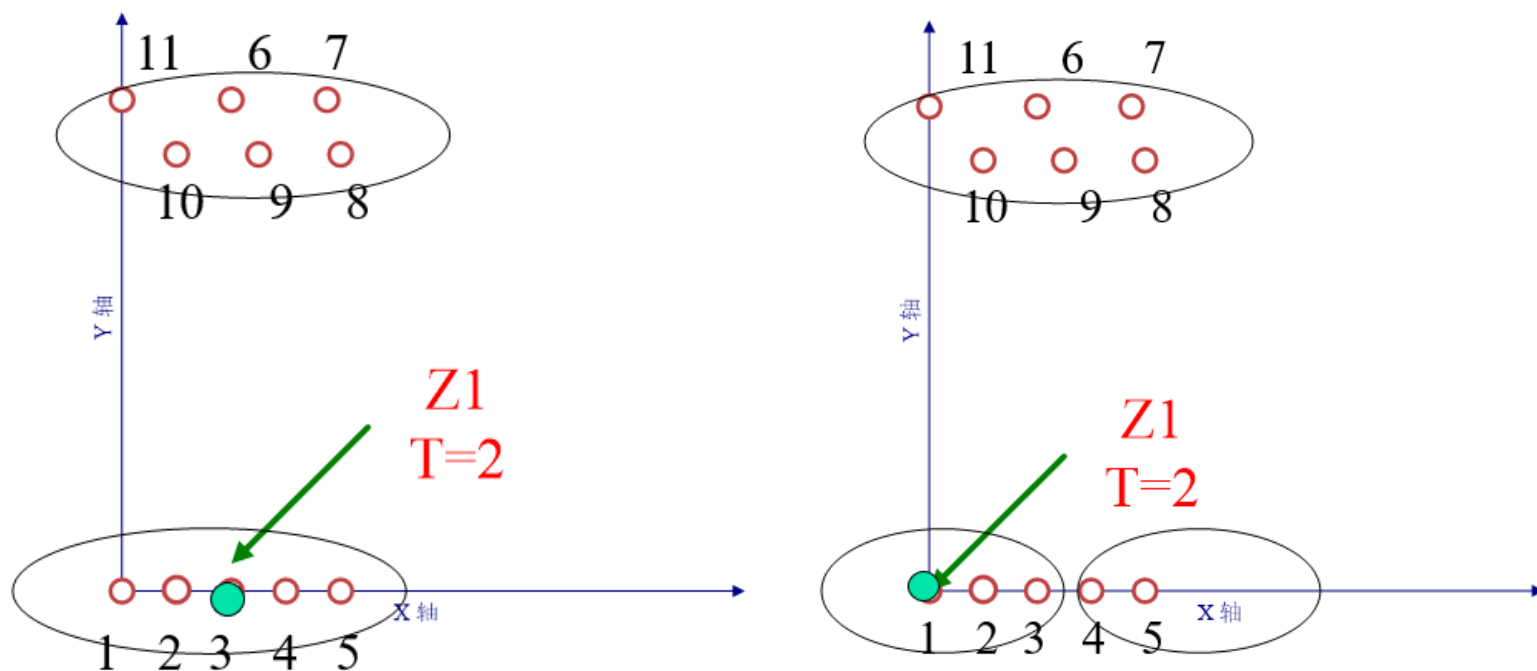
□ 按最近邻规则的简单试探法

- 在实际中，对于高维样本很难获得准确的先验知识，因此只能选用不同的阈值和起始点来试探，所以这种方法在很大程度上依赖于以下因素：
 - 第一个聚类中心的位置
 - 待聚类样本的排列次序
 - 距离阈值 T 的大小
 - 样本分布的几何性质

基于试探的聚类搜索算法

□ 按最近邻规则的简单试探法

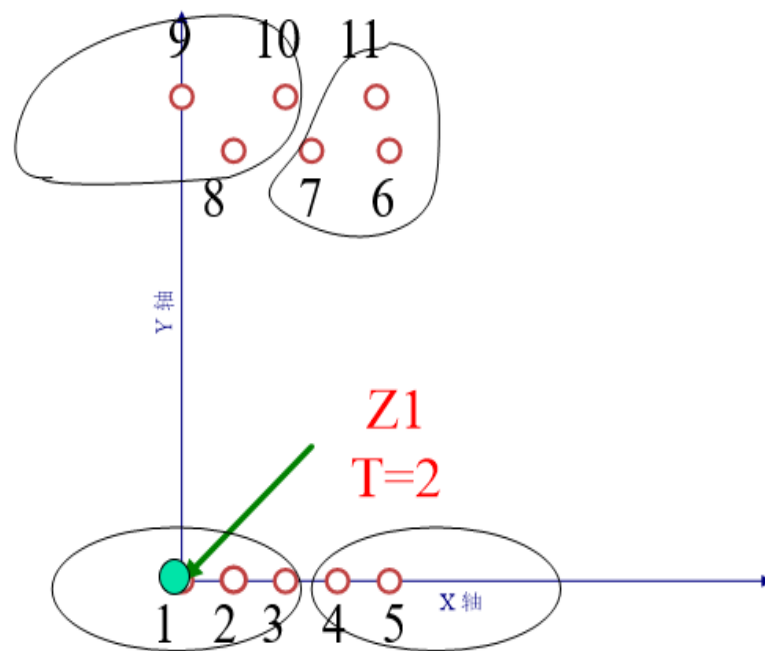
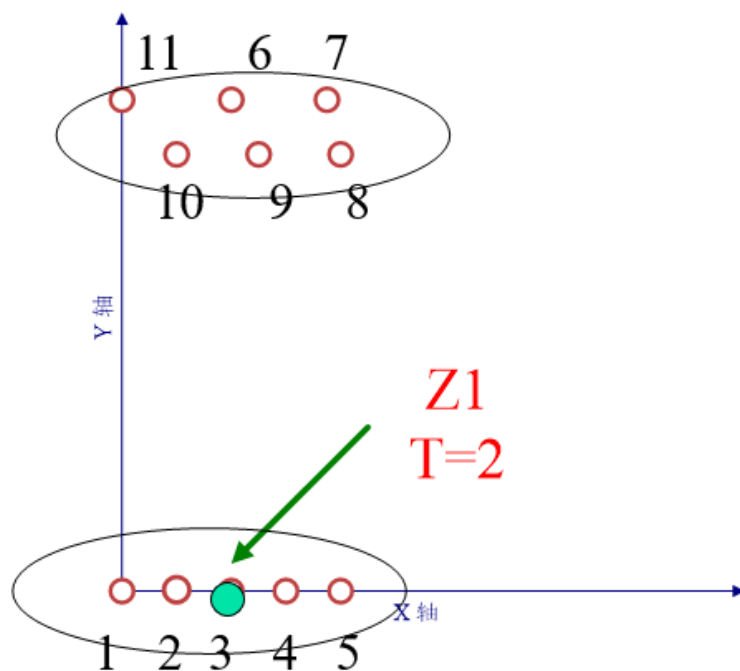
- 初始点不同



基于试探的聚类搜索算法

□ 按最近邻规则的简单试探法

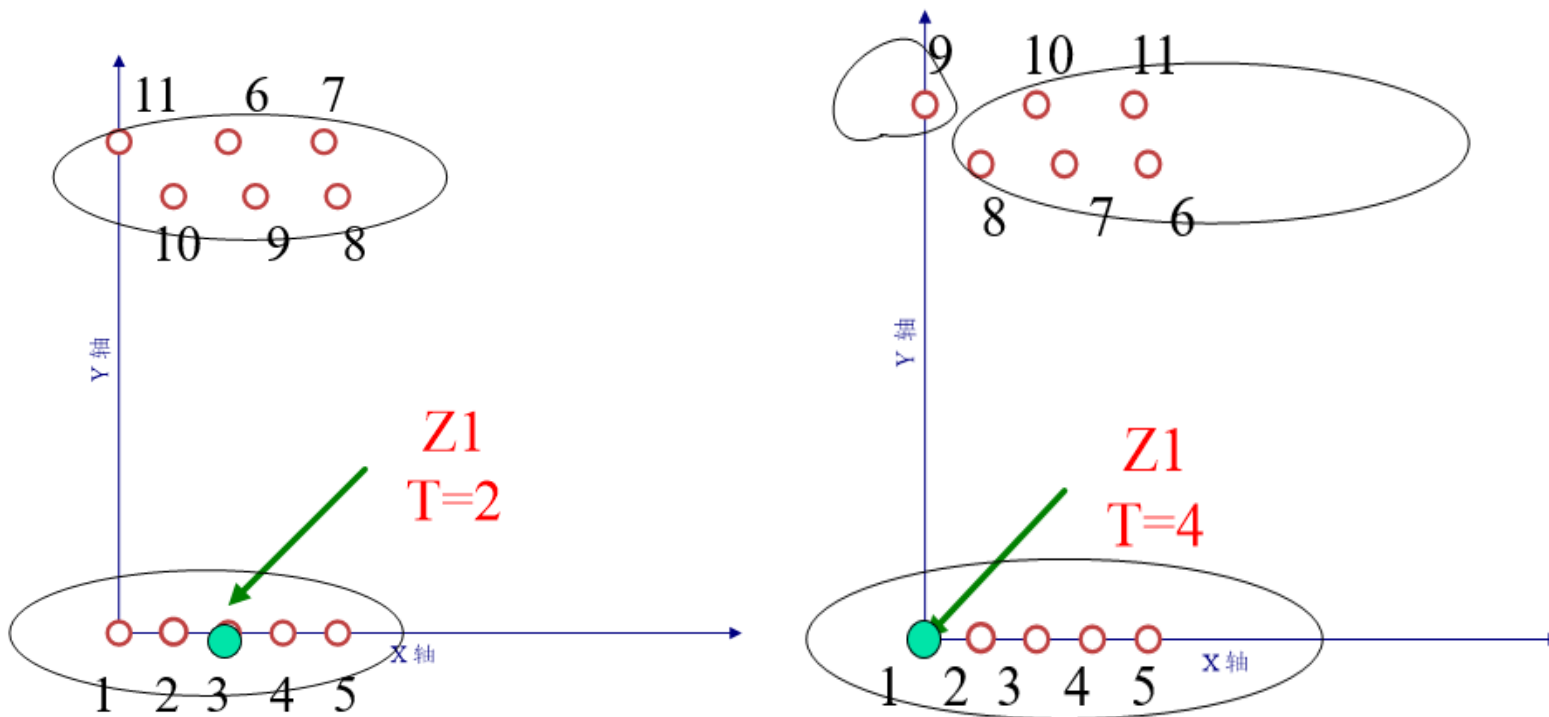
- 样本次序不同



基于试探的聚类搜索算法

□ 按最近邻规则的简单试探法

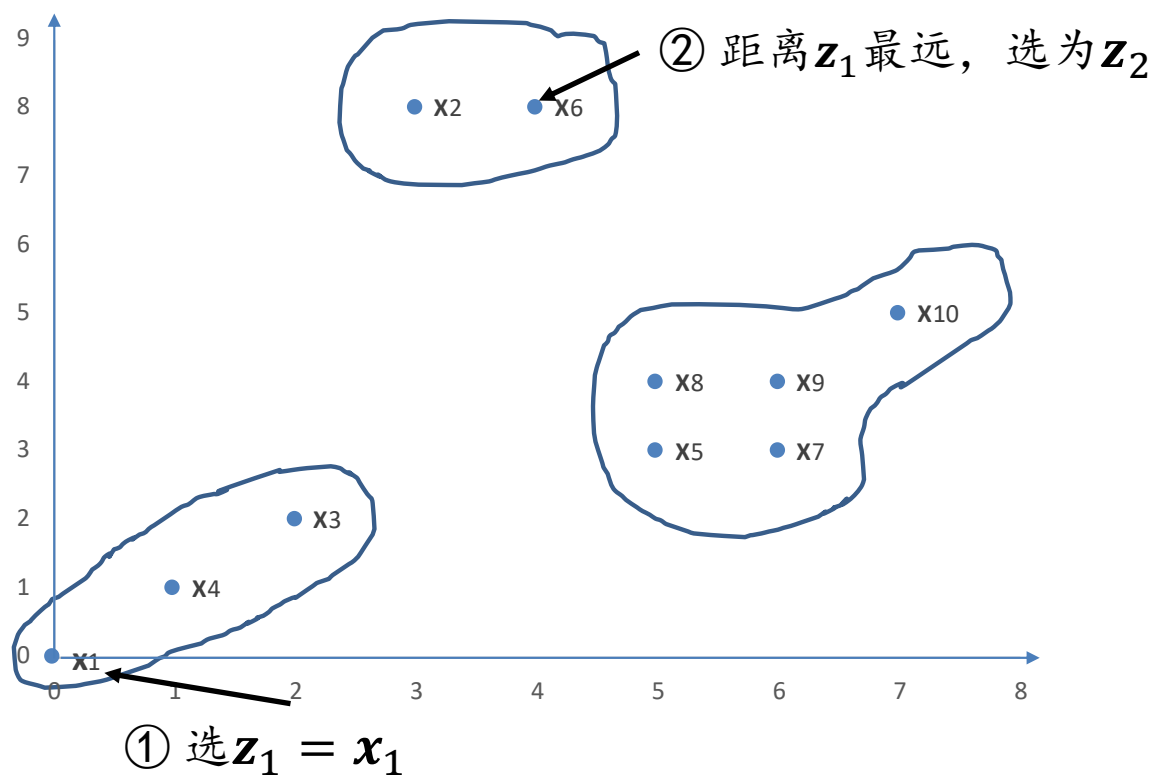
- 阈值 T 不同



基于试探的聚类搜索算法

□ 最大最小距离思想

- **基本思想**：以试探类间欧氏距离为最大作为预选出聚类中心的条件。



第一步：选任意一个样本作为第一个聚类中心，如 $z_1 = x_1$

第二步：选距离 z_1 最远的样本作为第二个聚类中心。

经计算， $\|x_6 - z_1\|$ 最大，所以 $z_2 = x_6$

第三步：逐个计算各样本 $\{x_i, i = 1, 2, \dots, N\}$ 与 $\{z_1, z_2\}$ 之间的距离，即

$$D_{i1} = \|x_i - z_1\|$$

$$D_{i2} = \|x_i - z_2\|$$

并选出其中的最小距离 $\min(D_{i1}, D_{i2})$, $i = 1, 2, \dots, N$

第四步：在所有样本的最小值中选出最大距离，若该最大值达到 $\|z_1 - z_2\|$ 的一定比例以上，则相应的样本点取为第三个聚类中心 z_3 ，即

若 $\max\{\min(D_{i1}, D_{i2}), i = 1, 2, \dots, N\} > \theta \|z_1 - z_2\|$ ，则 $z_3 = x_i$

否则，若找不到适合要求的样本作为新的聚类中心，则找聚类中心的过程结束。

这里， θ 可用试探法取一固定分数，如 $1/2$ 。

在此例中，当 $i=7$ 时，符合上述条件，故 $z_3 = x_7$

第五步：若有 z_3 存在，则计算 $\max\{\min(D_{i1}, D_{i2}, D_{i3}), i = 1, 2, \dots, N\}$ 。若该值超过 $\|z_1 - z_2\|$ 的一定比例，则存在 z_4 ，否则找聚类中心的过程结束。

在此例中，无 z_4 满足条件。

第六步：将模式样本 $\{x_i, i = 1, 2, \dots, N\}$ 按最近距离分到最近的聚类中心：

$z_1 = x_1$: $\{x_1, x_3, x_4\}$ 为第一类

$z_2 = x_6$: $\{x_2, x_6\}$ 为第二类

$z_3 = x_7$: $\{x_5, x_7, x_8, x_9, x_{10}\}$ 为第三类

最后，还可在每一类中计算各样本的均值，得到更具代表性的聚类中心。

系统聚类法

□ 基本思想

- 将数据样本按距离准则逐步分类，类别由多到少，直到获得合适的分类要求为止

□ 算法流程

第一步：设初始模式样本共有 N 个，每个样本自成一类，即建立 N 类， $G_1^{(0)}, G_2^{(0)}, \dots, G_N^{(0)}$ 。计算各类之间的距离（初始时即为各样本间的距离），得到一个 $N \times N$ 维的距离矩阵 $D^{(0)}$ 。这里，标号(0)表示聚类开始运算前的状态。

第二步：假设前一步聚类运算中已求得距离矩阵 $D^{(n)}$ ， n 为逐次聚类合并的次数，则求 $D^{(n)}$ 中的最小元素。如果它是 $G_i^{(n)}$ 和 $G_j^{(n)}$ 两类之间的距离，则将 $G_i^{(n)}$ 和 $G_j^{(n)}$ 两类合并为一类 $G_{ij}^{(n+1)}$ ，由此建立新的分类： $G_1^{(n+1)}, G_2^{(n+1)}, \dots$ 。

第三步：计算合并后新类别之间的距离，得 $D^{(n+1)}$ 。

计算 $G_{ij}^{(n+1)}$ 与其它没有发生合并的 $G_1^{(n+1)}, G_2^{(n+1)}, \dots$ 之间的距离，可采用多种不同的距离计算准则进行计算。

第四步：返回第二步，重复计算及合并，直到得到满意的分类结果。（如：达到所需的聚类数目，或 $D^{(n)}$ 中的最小分量超过给定阈值 D 等。）

系统聚类法

□ 距离准则函数

- 进行聚类合并的一个关键就是每次迭代中形成的聚类之间以及它们和样本之间距离的计算，采用不同的距离函数会得到不同的计算结果

□ 主要的距离计算准则：

- 最短距离法（两个集合所有距离最小值）
- 最长距离法（两个集合所有距离最大值）
- 类平均距离法（两个集合所有距离平均值）

动态聚类法

□ 基本思想

- 首先选择若干个样本点作为聚类中心，再按某种聚类准则（通常采用最小距离准则）使样本点向各中心聚集，从而得到初始聚类；
- 然后判断初始分类是否合理，若不合理，则修改聚类
- 如此反复进行修改聚类的迭代算法，直至合理为止。

□ 代表算法

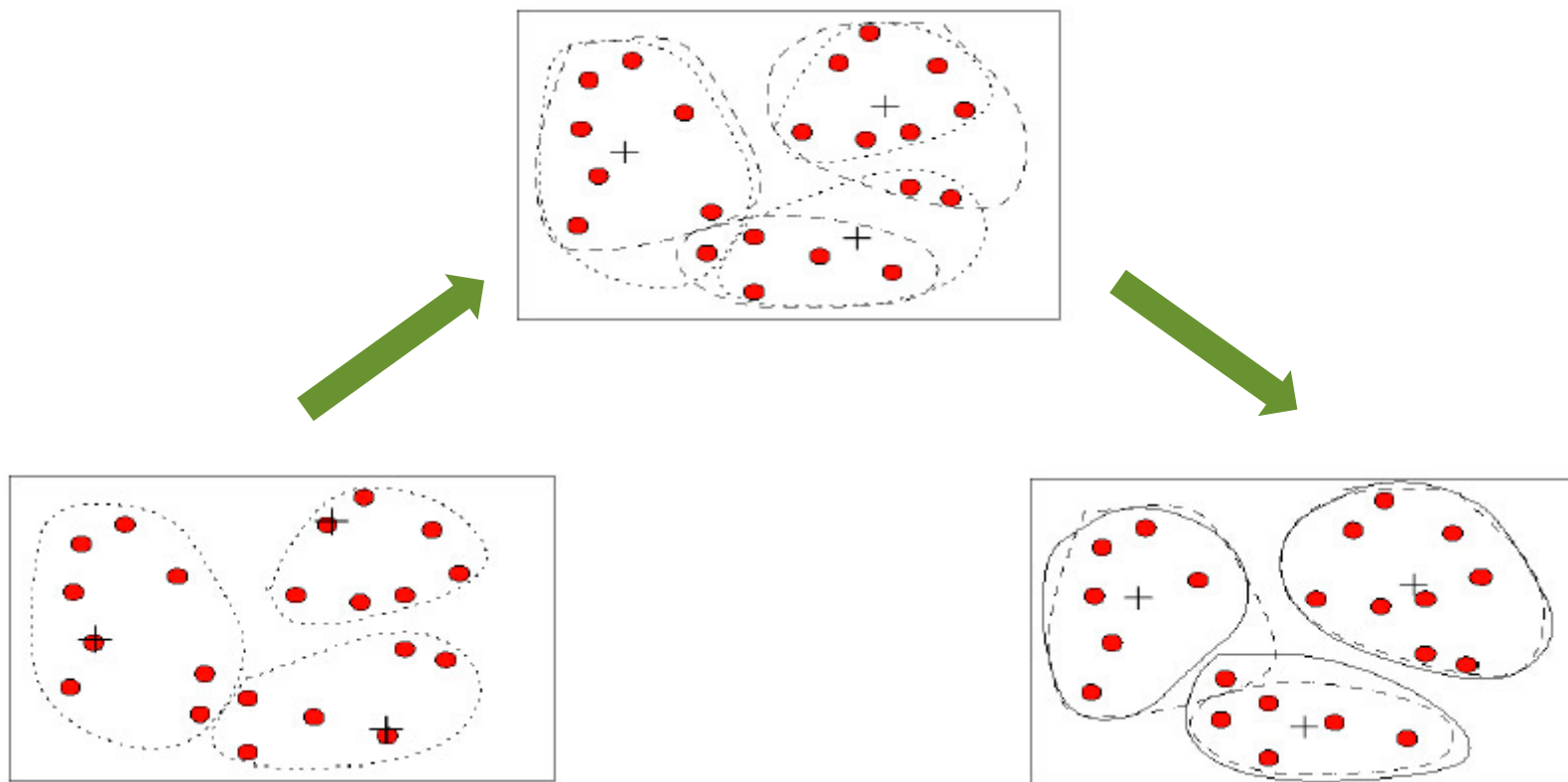
- K-means算法
- ISODATA算法（迭代自组织数据分析算法）

K-means算法

□ 算法流程

- Step1: 选择一个聚类数量 k
- Step2: 初始化聚类中心 $\mu_1, \dots, \mu_k$
- 随机选择 k 个样本点, 设置这些样本点为中心
- Step3: 对每个样本点, 计算样本点到 k 个聚类中心的距离 (使用某种距离度量) , 将样本点分距离它最近的聚类中心所属的聚类
- Step4: 重新计算聚类中心, 聚类中心为属于这一个个聚类的**所有样本的均值**
- Step5: 如果没发生样本所属的聚类改变的情况, 则退出, 否则, 返回Step3继续

K-means算法



K-means算法

□ 讨论

- K-means算法的结果受如下选择的影响：
 - 所选聚类的数目
 - 聚类中心的初始分布
 - 样本分布的几何性质
 -
- 在实际应用中，需要试探不同的K值和选择不同的聚类中心的起始值。
- 如果数据样本可以形成若干个相距较远的孤立的区域分布，一般都能得到较好的收敛效果。
- K-means算法比较适合于分类数目已知的情况。

K-means++算法

□ 基本思想: **K个初始聚类中心相互之间应该分得越开越好。**

K-means++算法

Step 1:从数据集中随机选取一个样本作为初始聚类中心 c_1 ;

Step 2:首先计算每个样本与当前已有聚类中心之间的最短距离（即与最近的一个聚类中心的距离），用 $D(x)$ 表示；接着计算每个样本被选为下一个聚类中心的概率 $\frac{D(x)^2}{\sum_{x \in X} D(x)^2}$ 。最后，按照轮盘法选择出下一个聚类中心；

Step 3:重复第2步直到选择出共 K 个聚类中心；

之后的过程与经典K-means算法中第2步至第4步相同

ISODATA算法

□ 迭代自组织数据分析算法

- Iterative Self-organizing Data Analysis Techniques

□ 基本步骤和思路

- (1) 选择某些初始值。可选不同的参数，也可在迭代过程中人为修改，以将N个样本按指标分配到各个聚类中心中去。
- (2) 计算各类中诸样本的距离指标函数。
- (3) ~ (6) 按给定的要求，将前一次获得的聚类集合进行分裂和合并处理，(5) 为分裂处理，(6) 为合并处理，从而获得新的聚类中心。
- (7) 重新进行迭代运算，计算各项指标，判断聚类结果是否符合要求。经过多次迭代后，若结果收敛，则运算结束。

ISODATA算法

□ 过程中能根据各个类别的实际情况进行**分裂**和**合并**两种操作来调整聚类中心数

ISODATA算法
Step 1: 从数据集中随机选取 K_0 个样本作为初始聚类中心 $C = \{c_1, c_2, \dots, c_{K_0}\}$;
Step 2: 针对数据集中每个样本 x_i ,计算它到 K_0 个聚类中心的距离并将其分到距离最小的聚类中心所对应的类中;
Step 3: 判断上述每个类中的元素数目是否小于 N_{min} 。如果小于 N_{min} 则需要丢弃该类,令 $K = K - 1$,并将该类中的样本重新分配给剩下类中距离最小的类;
Step 4: 针对每个类别 c_i ,重新计算它的聚类中心 $c_i = \frac{1}{ c_i } \sum_{x \in c_i} x$ (即属于该类的所有样本的质心) ;
Step 5: 如果当前 $K \leq \frac{K_0}{2}$,说明当前类别数太少,前往分裂操作;
Step 6: 如果当前 $K \geq 2K_0$,说明当前类别数太多,前往合并操作;
Step 7: 如果达到最大迭代次数则终止,否则回到第2步继续执行;

ISODATA算法

ISODATA-合并操作

Step 1:计算当前所有类别聚类中心两两之间的距离，用矩阵 D 表示，其中 $D(i, i) = 0$;

Step 2:对于 $D(i, j) < d_{min} (i \neq j)$ 的两个类别需要进行合并操作，变成一个新的类，该类的聚类中心位置为：

$$m_{new} = \frac{1}{n_i + n_j} (n_i m_i + n_j m_j)$$

上式中的 n_i 和 n_j 表示这两个类别中的样本个数，新的聚类中心可以看作是对这两个类别进行加权求和。如果其中一个类所包含的样本个数较多，所合成的新类就会更加偏向它。

两个类别对应聚类中心之间所允许最小距 d_{min} ：是否进行合并的阈值

ISODATA算法

ISODATA-分裂操作

Step 1:计算每个类别下所有样本在每个维度下的方差；

Step 2:针对每个类别的所有方差挑选出最大的方差 σ_{max} ；

Step 3:如果某个类别的 $\sigma_{max} > Sigma$ 并且该类别所包含的样本数量 $n_i \geq 2n_{min}$,则可以进行分裂操作，前往步骤4.如果不满足上述条件则退出分裂操作；

Step 4:将满足步骤3中条件的类分裂成两个子类别并令 $K = K + 1$ ， h 是一个比例系数

$$m_i^{(+)} = m_i + h \cdot \sigma_{max}, m_i^{(-)} = m_i - h \cdot \sigma_{max}.$$

最大方差 $Sigma$: 用于衡量某个类别中样本的分散程度。

当样本的分散程度超过这个值时，则有可能进行分裂操作

ISODATA算法

□ 与K-means算法比较

- K-means算法通常适合于类别数目已知的聚类，而ISODATA算法则更加灵活；
- 从算法角度看，ISODATA算法与K-means算法相似，聚类中心都是通过样本均值的迭代运算来决定的；
- ISODATA算法加入了一些试探步骤，并且可以结合人机交互的结构，使其能利用中间结果所取得的经验更好地进行分类；
- ISODATA原理非常直观，不过它需要额外指定较多的参数，并且某些参数同样很难准确指定出一个较合理的值，因此ISODATA算法在实际过程中并没有特别受欢迎。

大纲

- 相关概念
- 距离度量
- 聚类准则
- 聚类方法
- 聚类评价
- 前沿进展

聚类评价

□ 可考虑用以下几个指标来评价聚类效果

- 聚类中心之间的距离
 - 距离值大，通常可考虑分为不同类
- 聚类域中的样本数目
 - 样本数目少且聚类中心距离远，可考虑是否为噪声
- 聚类域内样本的距离方差
 - 方差过大的样本可考虑是否属于这一类

□ 讨论

- 聚类目前还没有一种通用的准则，往往需要根据实际应用来选择合适的方法

聚类评价

□ 外部指标 (external index)

- 将聚类结果与某个“参考模型” (reference model) 进行比较
- 如 Jaccard 系数, FM 指数, Rand 指数

□ 内部指标 (internal index)

- 直接考察聚类结果而不用任何参考模型
- 如 DB 指数, Dunn 指数等

□ 基本想法

- “簇内相似度” (intra-cluster similarity) 高, 并且
- “簇间相似度” (inter-cluster similarity) 低

聚类评价

□ 常用评价指标（**标签未知**）

- **Compactness (CP) 紧密度**

$$\overline{CP}_i = \frac{1}{|\Omega_i|} \sum_{x_i \in \Omega_i} \|x_i - w_i\| \quad \overline{CP} = \frac{1}{K} \sum_{k=1}^K \overline{CP}_k$$

- Ω_i 表示聚类得到的一个簇， w_i 表示该簇的中心， K 表示簇（类）的个数。 $\overline{CP}$ 值越小表示类内越紧凑
- 缺点：没有考虑类间聚类效果

聚类评价

□ 常用评价指标（**标签未知**）

- Separation (SP) 间隔度

$$\overline{SP} = \frac{2}{k^2 - k} \sum_{i=1}^k \sum_{j=i+1}^k \|w_i - w_j\|_2$$

- w_i 表示第*i*簇（类）的中心， w_j 表示第*j*簇（类）的中心。 $\overline{SP}$ 值越大表示类间越分散
- 缺点： 没有考虑类内聚类效果

聚类评价

□ 常用评价指标（标签未知）

- Davies-Bouldin Index (DBI) 戴维森堡丁指数/分类适确性指标

$$DB = \frac{1}{k} \sum_{i=1}^k \max_{j \neq i} \left(\frac{\bar{C}_i + \bar{C}_j}{\|w_i - w_j\|_2} \right)$$

- $\bar{C}_i$ 表示第*i*簇（类）的紧密度， w_i 表示第*i*簇（类）的中心。 DB 值越小，表示类内越紧凑，类间越分散
- 缺点：使用欧式距离，对于环状分布聚类评价很差

聚类评价

□ 常用评价指标（标签未知）

- Dunn Validity Index (DVI) 邓恩指数

$$DVI = \frac{\min_{0 < m \neq n < K} \left\{ \min_{\substack{\forall x_i \in \Omega_m \\ \forall x_j \in \Omega_n}} \{ \|x_i - x_j\| \} \right\}}{\max_{0 < m \leq K} \max_{\forall x_i, x_j \in \Omega_m} \{ \|x_i - x_j\| \}}$$

- x_i 表示簇 Ω_m 中第 i 个样本, x_j 表示簇 Ω_n 中第 j 个样本, 计算任意两个簇元素的最短距离（类间）除以任意簇中的最大距离（类内）
- 缺点：对离散点的聚类测评很高、对环状分布测评效果差

聚类评价

□ 常用评价指标 (标签已知)

- Cluster Accuracy (CA) 聚类准确率
- Rand index (RI) 兰德指数
- Adjusted Rand index (ARI) 调整兰德指数
- Mutual Information (MI) 互信息
- Normalized Mutual Information (NMI) 归一化互信息

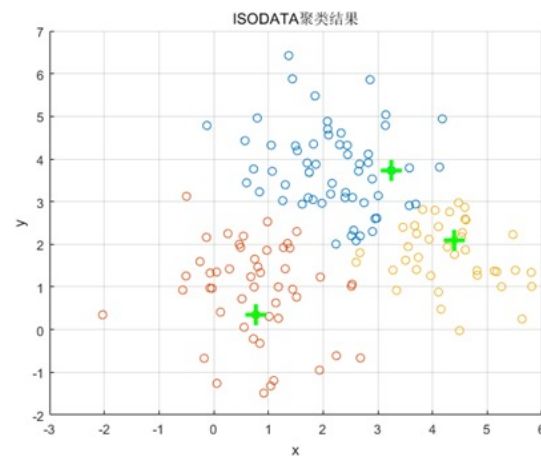
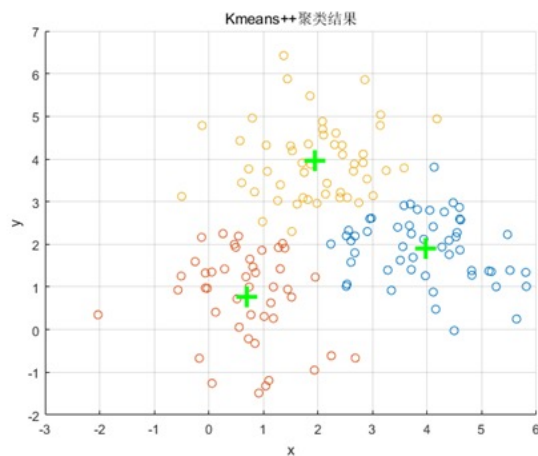
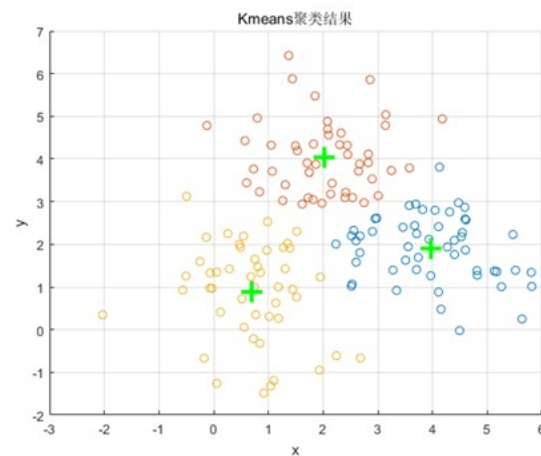
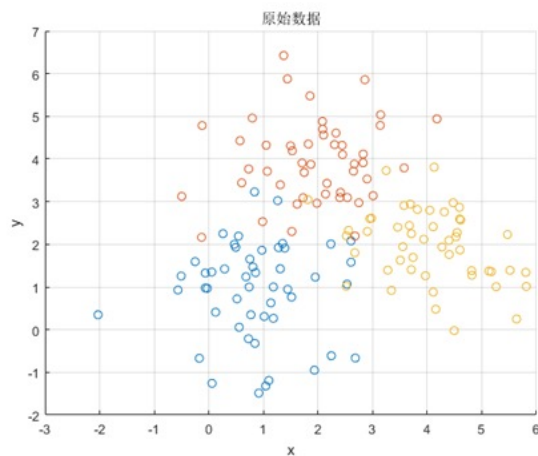
聚类评价

□ 推荐阅读

Fahad A, Alshatri N, Tari Z, et al. **A survey of clustering algorithms for big data: Taxonomy and empirical analysis**[J]. IEEE transactions on emerging topics in computing, 2014, 2(3): 267-279.

聚类评价

□ 可视化：二维高斯分布的数据聚类结果



大纲

- 相关概念
- 距离度量
- 聚类准则
- 聚类方法
- 聚类评价
- 前沿进展

前沿进展

□ 深度学习时代的聚类算法

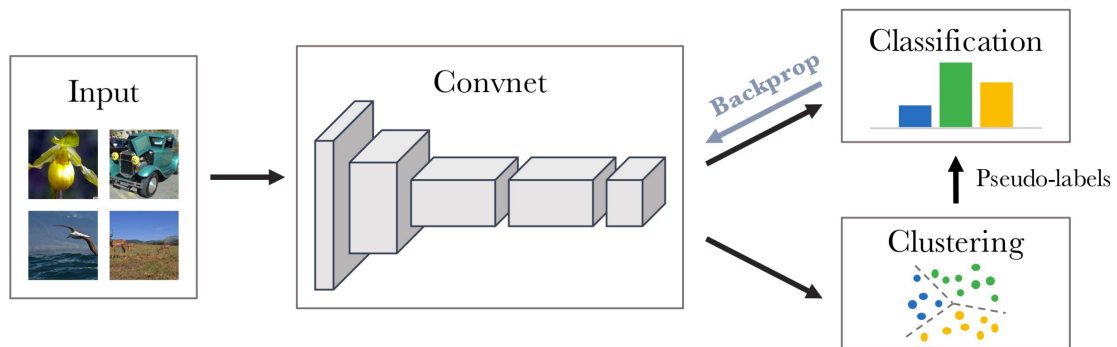


Fig. 1: Illustration of the proposed method: we iteratively cluster deep features and use the cluster assignments as pseudo-labels to learn the parameters of the convnet.

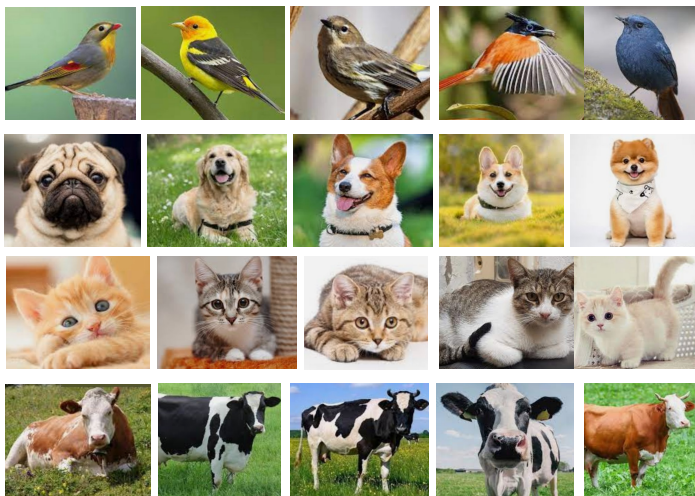


[1] Deep Clustering for Unsupervised Learning of Visual Features. ECCV 2018.

[2] Self-labelling via Simultaneous Clustering and Representation Learning. ICLR 2020.

监督学习的成功

监督学习

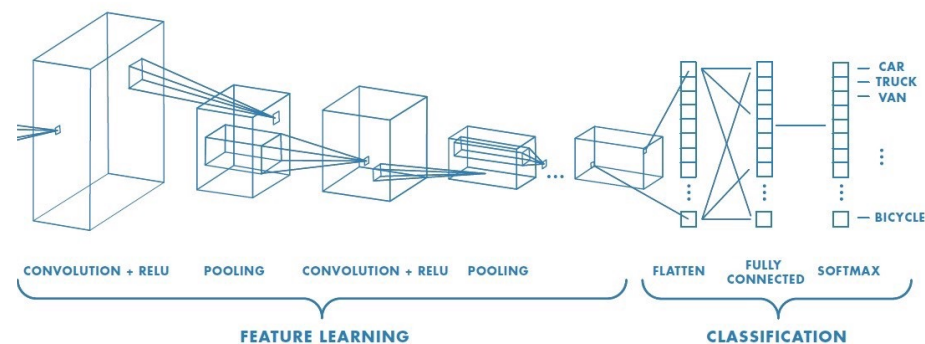


Bird

Dog

Cat

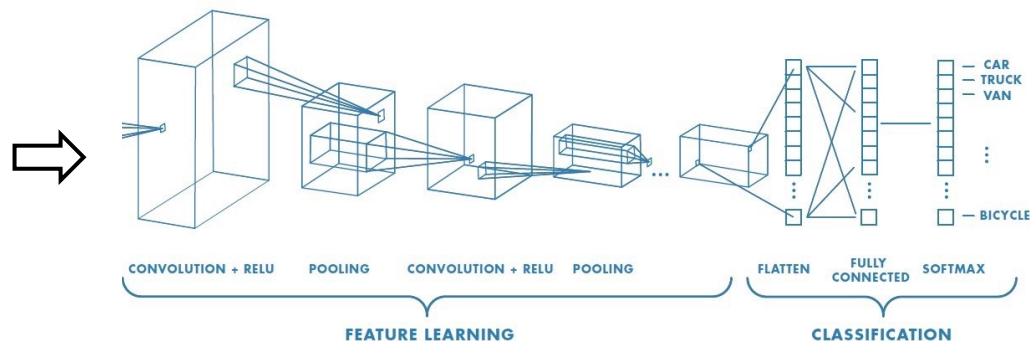
Cow



目标识别

监督学习的成功

监督学习



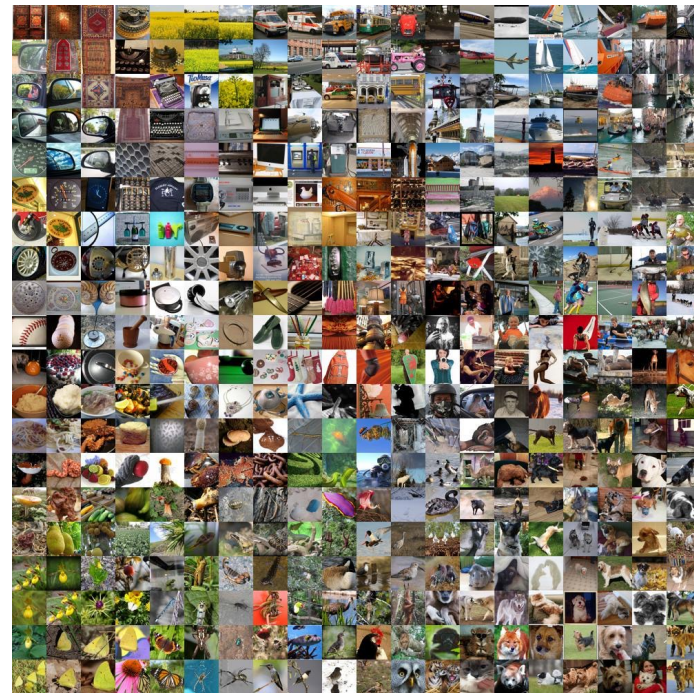
目标识别

□ 监督深度学习是如何成功的？

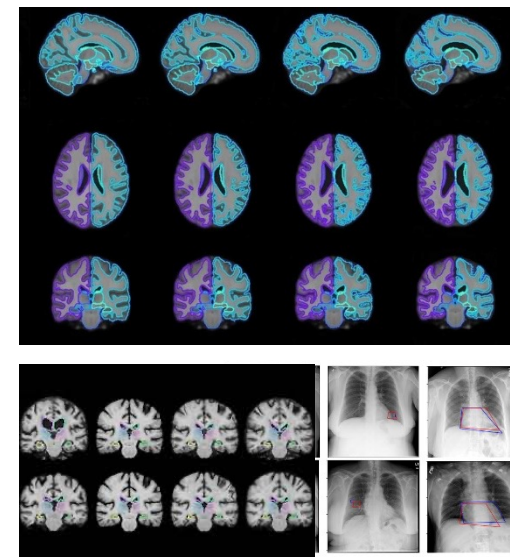
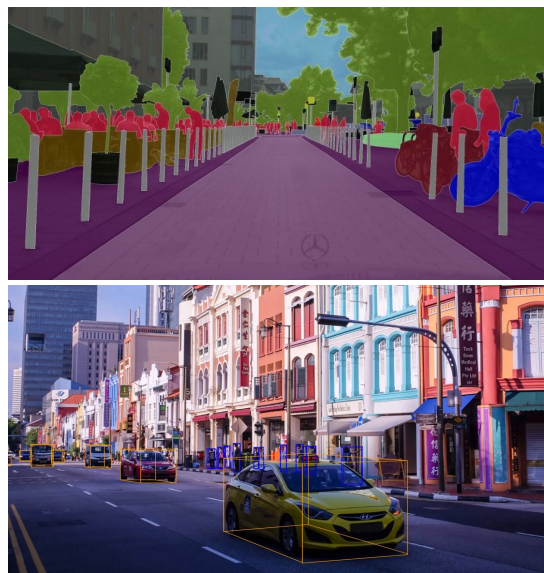
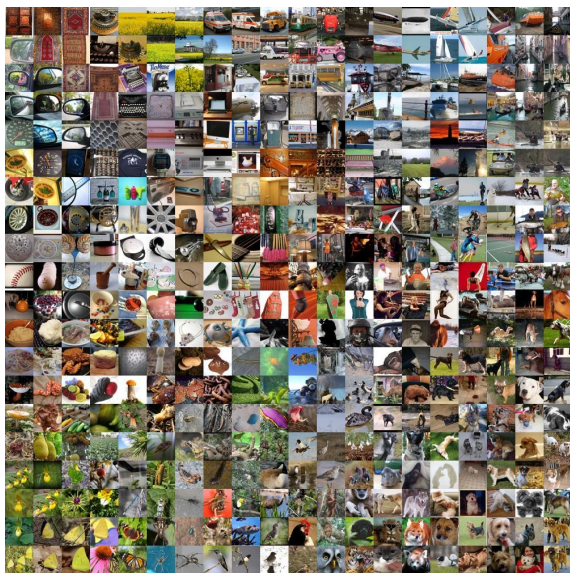
- 预先定义许多视觉的概念及类别进行学习
- 为每个概念类别收集大量、具有差异性的样本
- 对收集到的数据进行清洗和精细标注
- 采用多块GPU显卡训练几个小时甚至几天

现实场景的困境

- 数据体量大
- 数据标注时间长
- 数据标注代价高



如何有效利用原始未标注的数据



- 现实场景中，获取原始未标注的数据往往是**比较容易**的！
- 典型的监督学习技术**不能利用**这些未标注数据！
- 监督信号有时候会使得深度模型**变得有偏**！

自监督学习来解决以上困境?

什么是自监督学习

- 无监督学习的一种形式，其中数据没有提供（人类标注的）监督信息

什么是自监督学习

- 无监督学习的一种形式，其中数据没有提供（人类标注的）监督信息
- 通常需要定义一个前置（借口）任务让网络学习我们关心的事情

什么是自监督学习

- 无监督学习的一种形式，其中数据没有提供（人类标注的）监督信息
- 通常需要定义一个前置（借口）任务让网络学习我们关心的事情
- 对于大部分前置任务，我们需要保留一部分数据，让网络学会预测

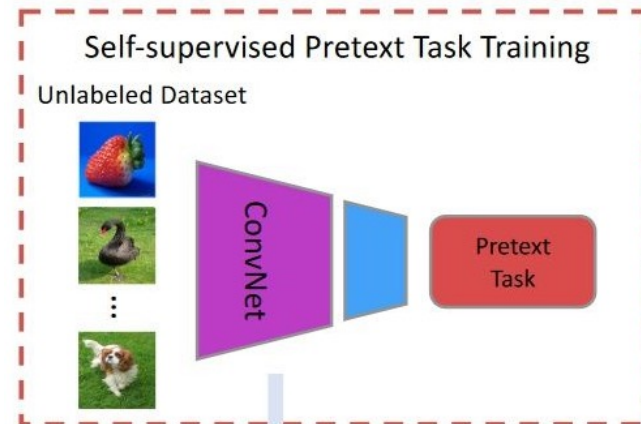
什么是自监督学习

- 无监督学习的一种形式，其中数据没有提供（人类标注的）监督信息
- 通常需要定义一个前置（借口）任务让网络学习我们关心的事情
- 对于大部分前置任务，我们需要保留一部分数据，让网络学会预测
- 通过前置任务学习到的特征会被用到不同的下游任务（通常包含标注）

什么是自监督学习

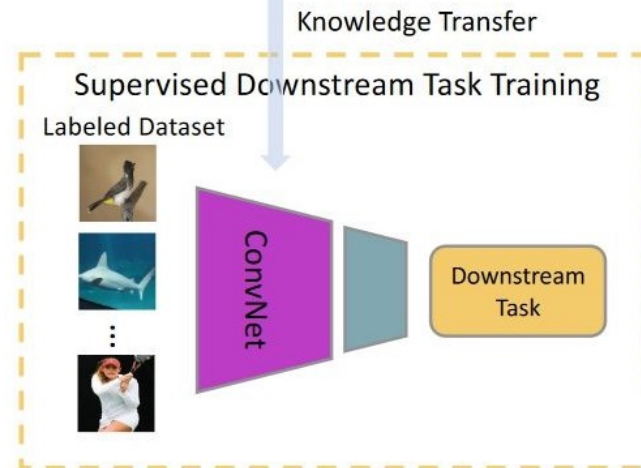
□ 自监督预训练

- 无标签数据
- 前置任务预训练 (Pretext task pre-training)



□ 下游任务迁移

- 有标签数据
- 训练测试数据集可不同
- 多种下游任务测试 (分类、分割、检测)



自监督学习分类

□ 前置任务学习 (Pretext task learning)

- 生成式方法：图像着色、图像修复 ...
- 判别式方法：图像拼图、图像旋转 ...

□ 对比学习 (Contrastive learning)

- CMC、MoCo、SimCLR、CO2 ...

□ 非对比学习 (Non-contrastive learning)

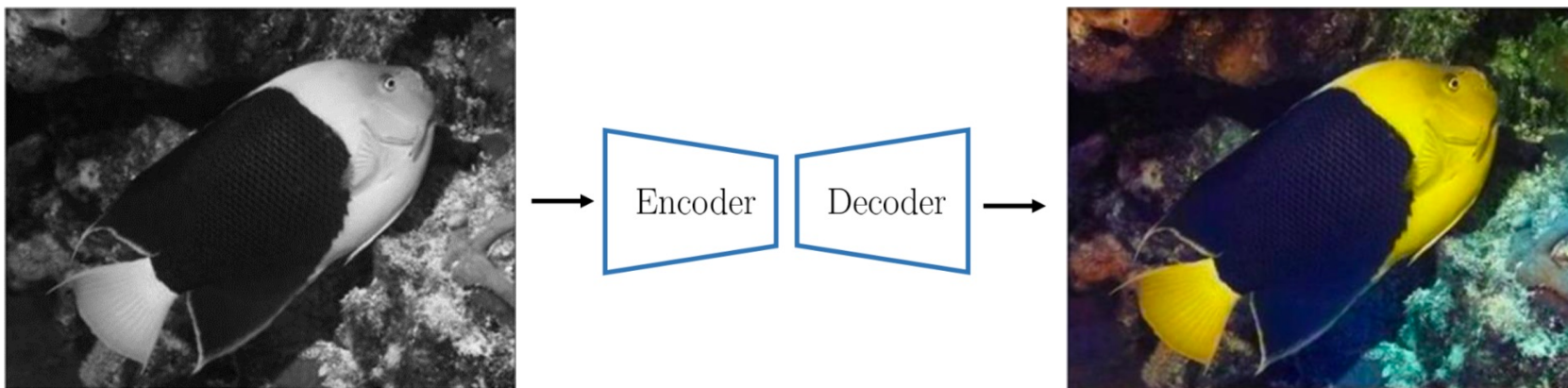
- 聚类方法：DeepCluster、SwAV ...
- 蒸馏方法：BYOL、SimSiam ...
- 信息最大化方法：W-MSE、Barlow Twins ...

前置任务学习

Hand-crafted pretext task learning

前置任务学习

□ 生成式方法—图像着色 (Image Colorization)



每个像素的颜色是什么？

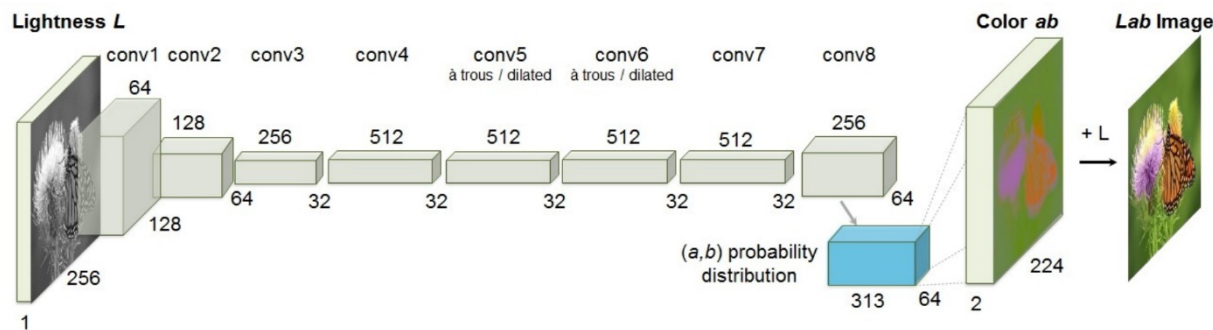
如果你不认识这个目标，这个任务将很难！

前置任务学习

□ 生成式方法—图像着色 (Image Colorization)

• 动机

- 尽管缺失了颜色信息，但是依旧可以理解灰度图像中的粗粒度语义内容
- 建立一个从灰度图像到彩色图像的映射，相当于从粗粒度语义学习细粒度语义
- 在学习的过程中虽然不知道图像类别，但网络掌握了语义信息



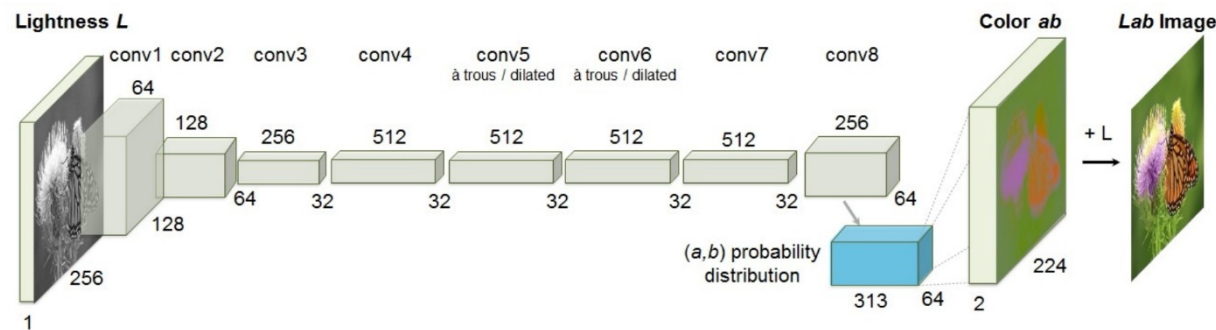
Zhang R, Isola P, Efros A A. Colorful image colorization. *ECCV 2016*

前置任务学习

□ 生成式方法—图像着色 (Image Colorization)

• 方法

- 首先把彩色图像转为灰度图像，把图像从RGB彩色空间转为Lab彩色空间（L是亮度，ab是色彩）
- 灰度图像L送入VGG网络，网络输出与重建目标ab构成损失
- 损失并没有使用MSE（回归任务），而是把每个像素点构成一个分类任务（ab彩色空间313个量化的位置块，即313类），使用加权的交叉熵分类损失

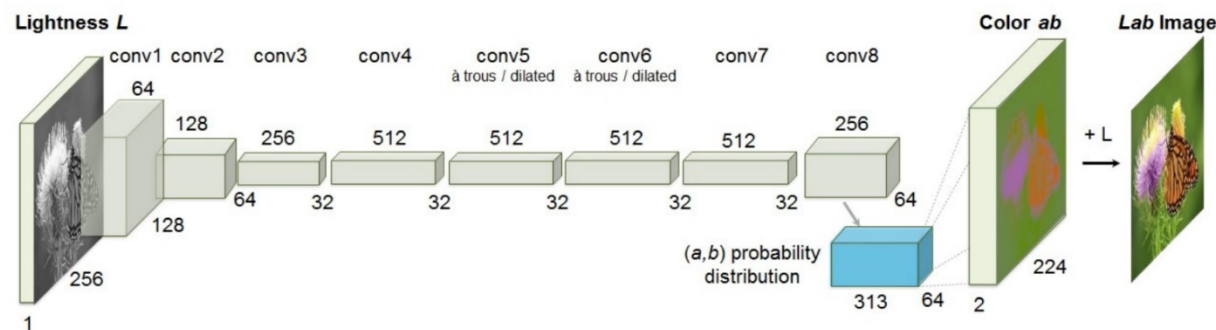


Zhang R, Isola P, Efros A A. Colorful image colorization. *ECCV 2016*

前置任务学习

□ 生成式方法—图像着色 (Image Colorization)

- 方法框架

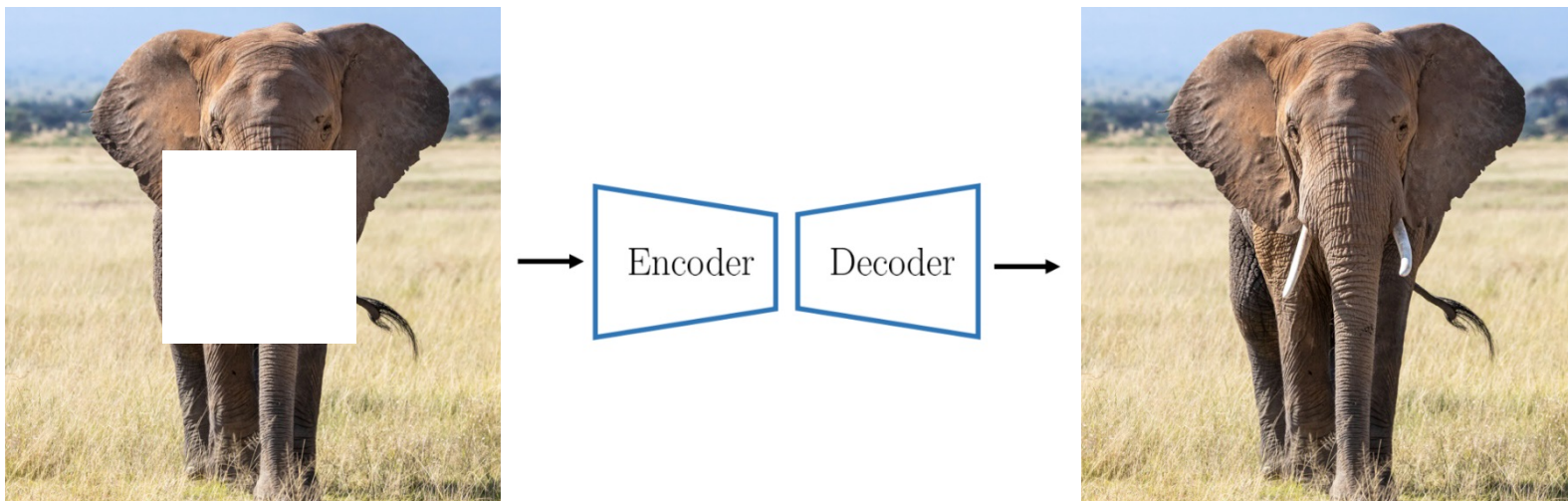


- 损失函数 (加权交叉熵分类损失)

$$L_{cl}(\hat{\mathbf{Z}}, \mathbf{Z}) = - \sum_{h,w} v(\mathbf{Z}_{h,w}) \sum_q \mathbf{Z}_{h,w,q} \log(\hat{\mathbf{Z}}_{h,w,q})$$

前置任务学习

□ 生成式方法—图像修复 (Image Inpainting)



需要保留细粒度信息以及上下文感知的能力

通常，重建任务是比较难**而且是**不确定的****

前置任务学习

□ 生成式方法—图像修复 (Image Inpainting)

• 动机

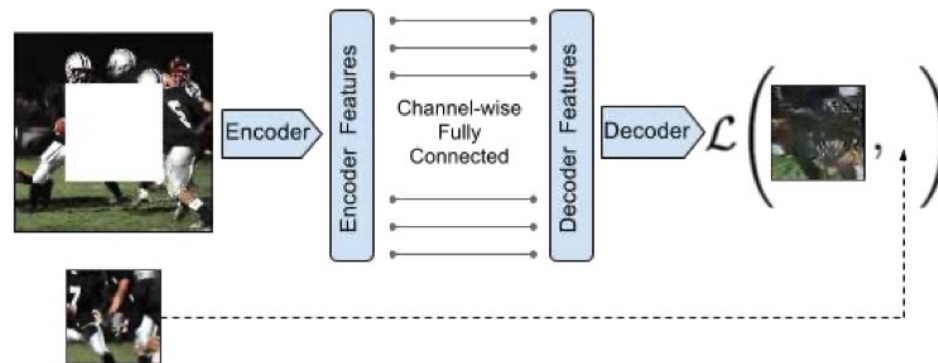
- 对一幅图像，遮挡住其中一部分，人可以根据未被遮挡的部分推测出遮挡区域的内容
- 对于一段英文，我们丢弃其中一些单词，可以根据上下文语境推理出这个单词
- 基于这样的上下文理解，在计算机视觉中，让网络推理被遮挡部分的内容，网络为了达到这个要求，它必须理解整幅图像的内容（掌握了语义）。

前置任务学习

□ 生成式方法—图像修复 (Image Inpainting)

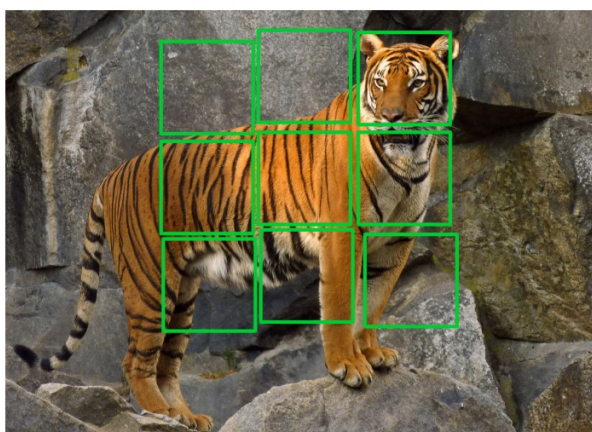
• 方法

- 给出一幅图像，采用某种策略遮挡图像的部分内容
- 把被遮挡的图像送入卷积神经网络，经过编码和解码得到和原图一样大小的输出
- 对遮挡部分计算MSE重建损失；对重建图和原图计算对抗损失，使重建图像和原图更像，两个损失按权重相加反向传播

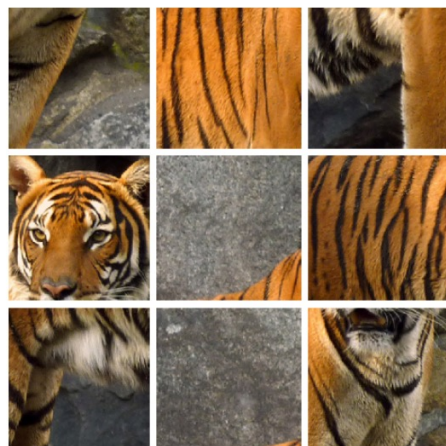


前置任务学习

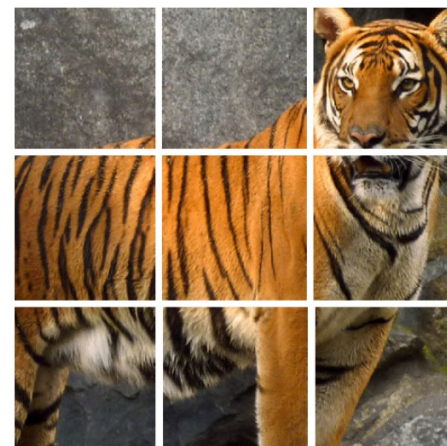
□ 判别式方法—图像拼图 (Jigsaw Puzzles)



(a)



(b)



(c)

把图像裁剪出不同的图像块、打乱、然后让网络学习拼图的能力高
拼图可以教会网络一个目标物体由哪几部分构成，每部分分别是什么

前置任务学习

□ 判别式方法—图像拼图 (Jigsaw)

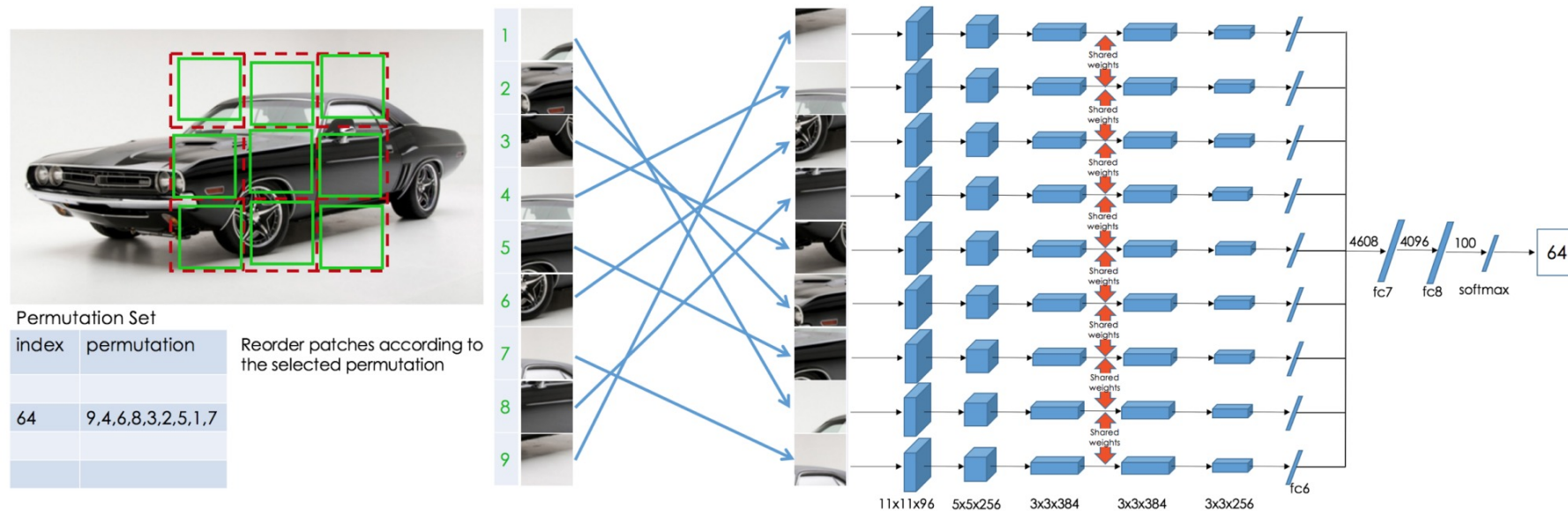
• 动机

- 一副完整的图像被拆成许多小块，根据一个模板拼回原来的样子
- 一方面依据整体图像的语义理解，另一方面还要依据一些细节特征（边缘、颜色等）
- 让网络完成拼图任务，一方面要让网络根据语义理解拼图，另一方面要避免网络走捷径
- 图像拼图在2D/3D/Video（打乱帧的正确顺序）都可以应用，是个不错的前置任务

前置任务学习

❑ 判别式方法—图像拼图 (Jigsaw)

• 方法框架



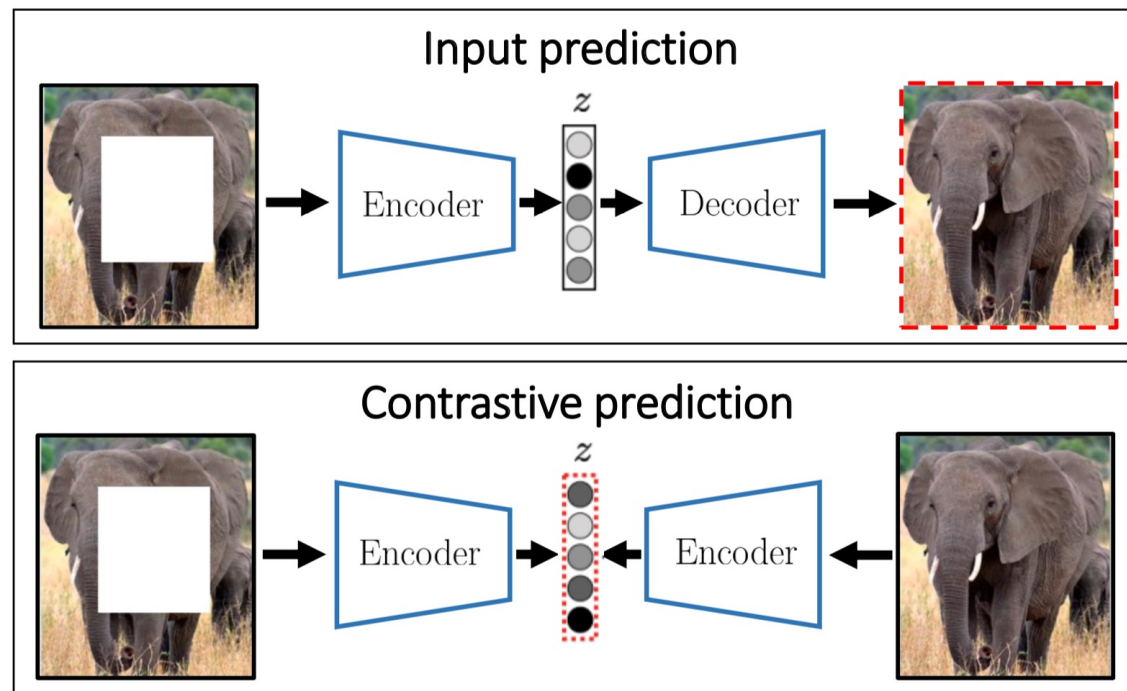
完成一个排列组合的问题

对比学习

Contrastive learning

对比学习

- 不再显示地手工构造前置任务
- 拉近同一张图像不同视图的距离，推远与其他图像之间的距离
- 一般通过数据增广，构造不同视图
- 难以保证负样本真实语义与正样本不同



对比学习

□ MoCo

• 动机

- 把对比学习看作一个字典查询问题，一个query样本经过编码器得到query表示，一个记忆库中有着许多字典，这些由被另一个编码器生成
- 如果字典中有一个key和query是相似的就被认为是正样本对，需要让其更相似，其他不相似的key和query构成负样本对，使其更远离
- 最终用对比损失优化模型

对比学习

□ MoCo

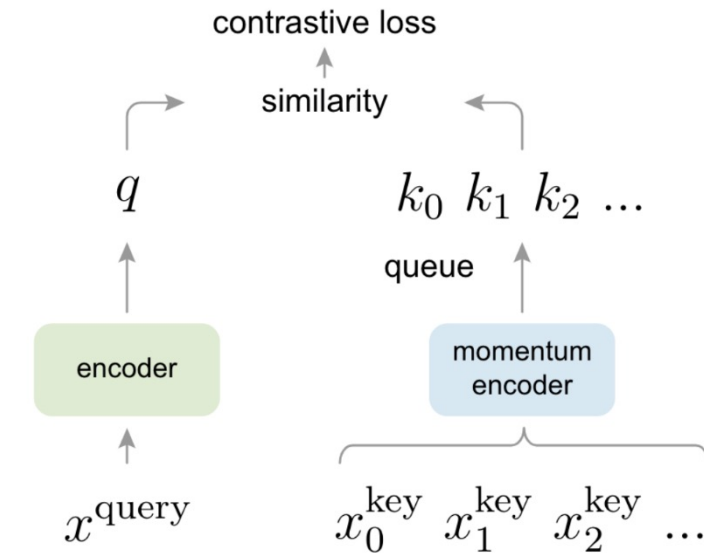
• 方法

- 一个查询编码器和一个动量编码器，动量编码器的权重由动量编码器过去的权重和查询编码器进行动量更新
- 一个batch的图像前向传播经过查询编码器和动量编码器，分别得到查询编码和队列编码，队列编码中的表征构成一个记忆库，被用过的编码不会马上丢失，而是放进队列里，队列“先进先出”
- 查询编码和队列编码中来自相同图像增广版本的被认为是正样本对，其他所有均为负样本

对比学习

MoCo

方法框架



$$\mathcal{L}_q = -\log \frac{\exp(q \cdot k_+ / \tau)}{\sum_{i=0}^K \exp(q \cdot k_i / \tau)}$$

Algorithm 1 Pseudocode of MoCo in a PyTorch-like style.

```
# f_q, f_k: encoder networks for query and key
# queue: dictionary as a queue of K keys (CxK)
# m: momentum
# t: temperature

f_k.params = f_q.params # initialize
for x in loader: # load a minibatch x with N samples
    x_q = aug(x) # a randomly augmented version
    x_k = aug(x) # another randomly augmented version

    q = f_q.forward(x_q) # queries: NxK
    k = f_k.forward(x_k) # keys: NxK
    k = k.detach() # no gradient to keys

    # positive logits: Nx1
    l_pos = bmm(q.view(N,1,C), k.view(N,C,1))

    # negative logits: NxK
    l_neg = mm(q.view(N,C), queue.view(C,K))

    # logits: Nx(1+K)
    logits = cat([l_pos, l_neg], dim=1)

    # contrastive loss, Eqn. (1)
    labels = zeros(N) # positives are the 0-th
    loss = CrossEntropyLoss(logits/t, labels)

    # SGD update: query network
    loss.backward()
    update(f_q.params)

    # momentum update: key network
    f_k.params = m*f_k.params+(1-m)*f_q.params

    # update dictionary
    enqueue(queue, k) # enqueue the current minibatch
    dequeue(queue) # dequeue the earliest minibatch
```

bmm: batch matrix multiplication; mm: matrix multiplication; cat: concatenation.

对比学习

□ SimCLR

- 动机

- 在特征空间，正样本对相似，负样本对远离
- 正样本对就用同一个图片的不同数据增强构造，负样本对就是不同的图片
- 直接基于同一个batch构造正负样本，无需额外存储空间暂存样本对（如MoCo）

对比学习

□ SimCLR

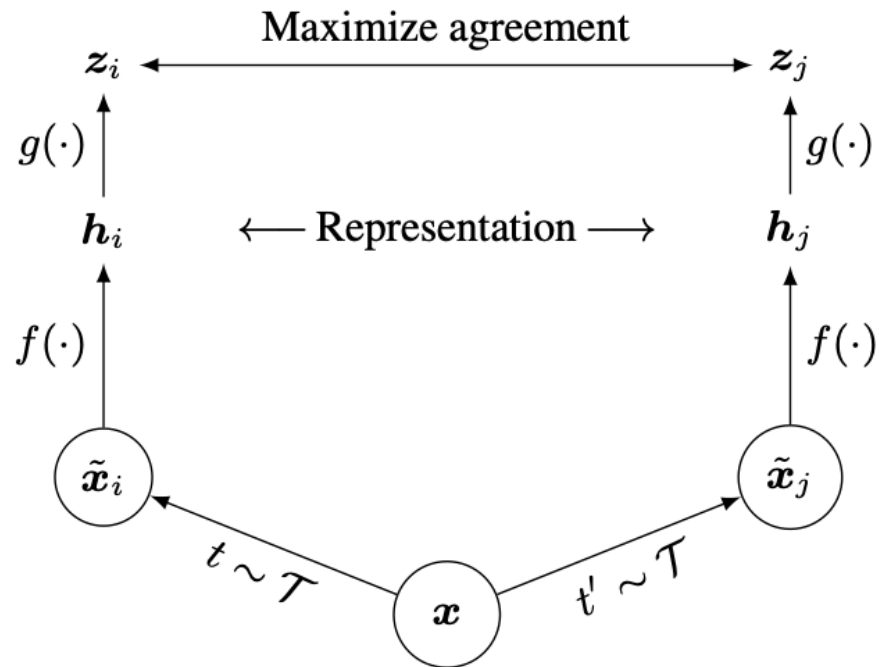
• 方法

- 对batch大小为N的一批图像进行增强，每个图像 x 采用两次随机数据增强得到 x_{2i} 和 x_{2i-1} ，被认为是正样本对（一共N对），不同图像构成负对
- 经过同一个CNN网络得到特征表示向量
- 在经过一个映射头映射到较低的维度（128维），并对表示归一化
- 使用infoNCE损失，让正对相似，负对远离

对比学习

□ SimCLR

- 方法框架



$$\ell_{i,j} = -\log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(\text{sim}(z_i, z_k)/\tau)}$$

对比学习

□ SimCLR

• 总结

- 数据增强方式的探索：颜色变换和裁剪变换一起用效果最好
- 通过实验验证了多层投影头非常重要
- 缺点：这个方法需要很大的batchsize，设置到了4096，显然平民玩家是玩不起的

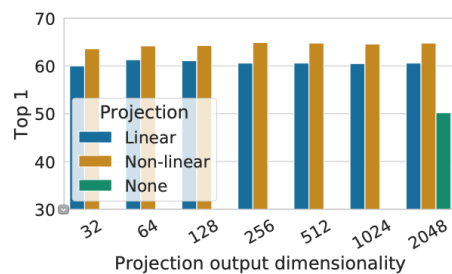


Figure 8. Linear evaluation of representations with different projection heads $g(\cdot)$ and various dimensions of $z = g(h)$. The representation h (before projection) is 2048-dimensional here.

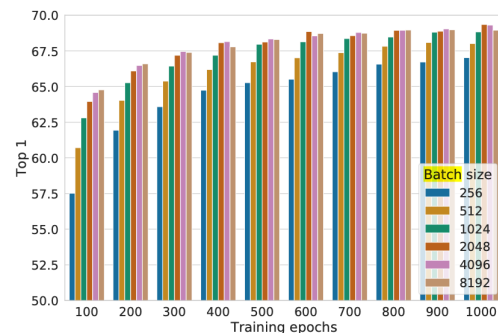
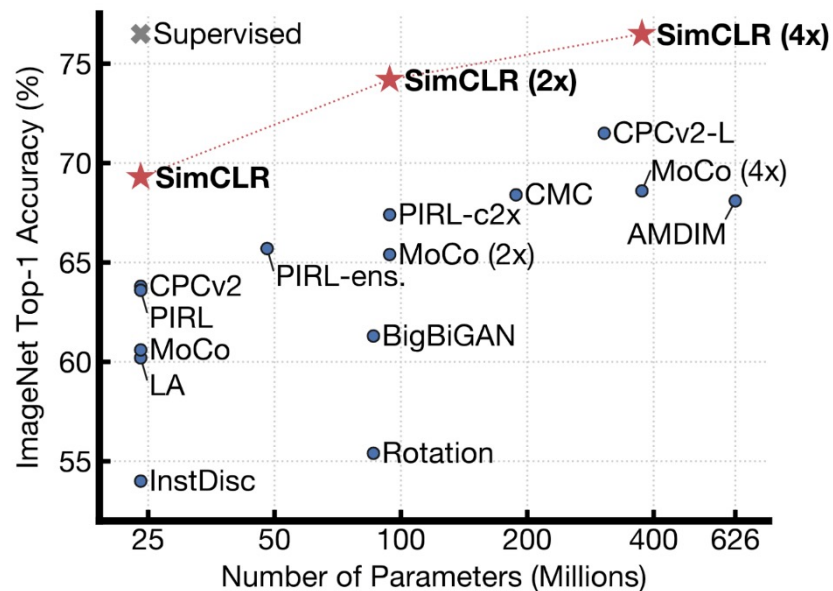


Figure 9. Linear evaluation models (ResNet-50) trained with different batch size and epochs. Each bar is a single run from scratch.¹⁰

对比学习

□ SimCLR

• 实验结果



Method	Architecture	Param (M)	Top 1	Top 5
<i>Methods using ResNet-50:</i>				
Local Agg.	ResNet-50	24	60.2	-
MoCo	ResNet-50	24	60.6	-
PIRL	ResNet-50	24	63.6	-
CPC v2	ResNet-50	24	63.8	85.3
SimCLR (ours)	ResNet-50	24	69.3	89.0
<i>Methods using other architectures:</i>				
Rotation	RevNet-50 (4×)	86	55.4	-
BigBiGAN	RevNet-50 (4×)	86	61.3	81.9
AMDIM	Custom-ResNet	626	68.1	-
CMC	ResNet-50 (2×)	188	68.4	88.2
MoCo	ResNet-50 (4×)	375	68.6	-
CPC v2	ResNet-161 (*)	305	71.5	90.1
SimCLR (ours)	ResNet-50 (2×)	94	74.2	92.0
SimCLR (ours)	ResNet-50 (4×)	375	76.5	93.2

Table 6. ImageNet accuracies of linear classifiers trained on representations learned with different self-supervised methods.

非对比学习

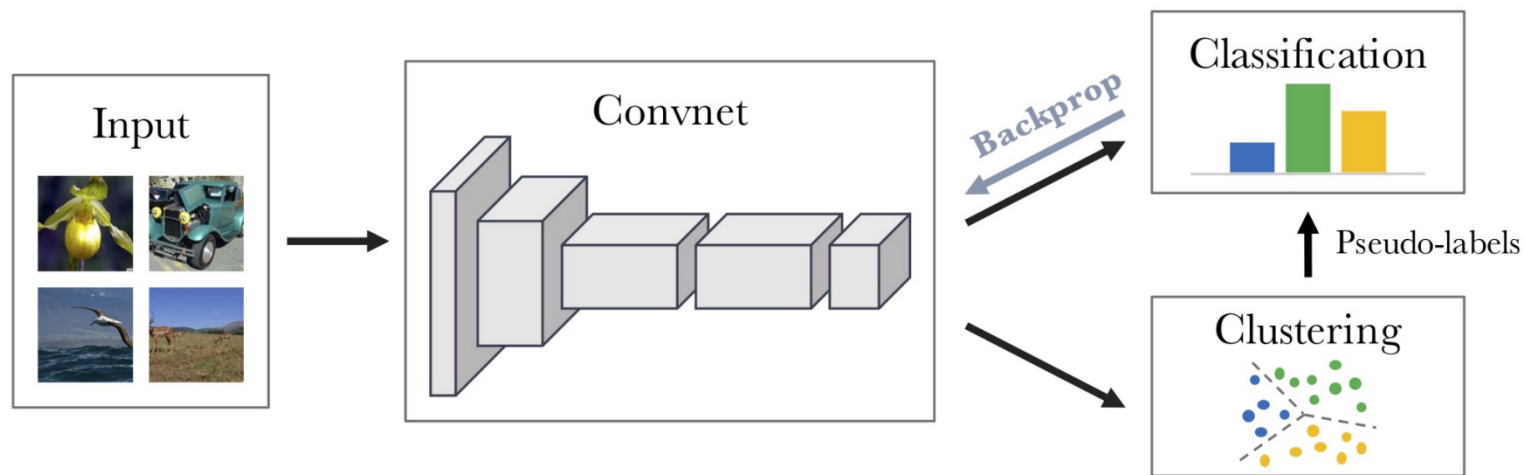
Non-Contrastive learning

非对比学习

□ 聚类方法—DeepCluster

• 动机

- 将经典的无监督聚类和深度学习结合进行端到端训练

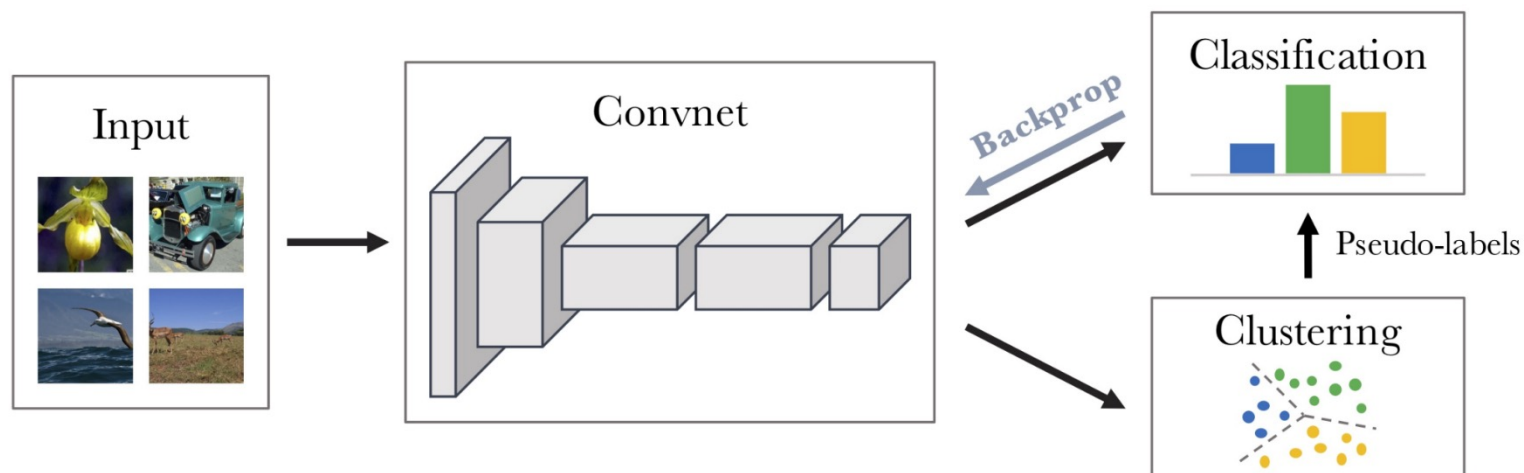


非对比学习

□ 聚类方法—DeepCluster

• 方法

- 对提取的特征进行无监督聚类（如k-means）
- 聚类后对样本打上伪标签，再训练网络预测伪标签



非对比学习

□ 聚类方法—SwAV

- 动机

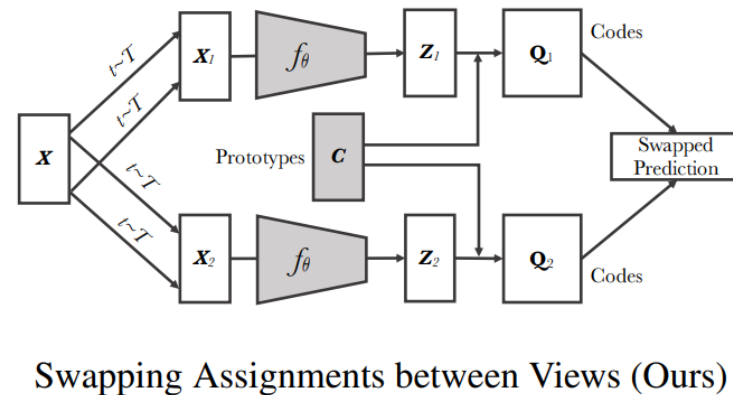
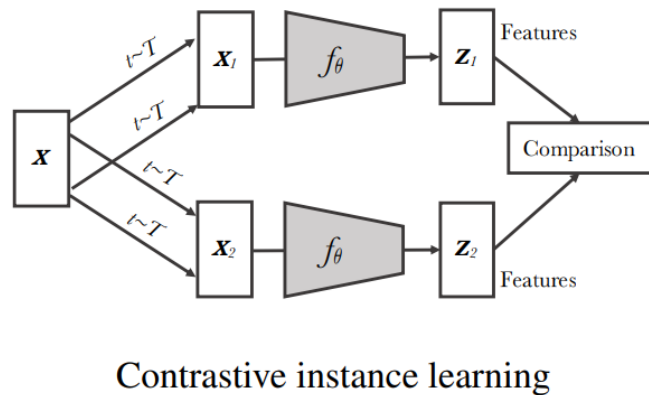
- 使用聚类技术解决对比学习中的对比问题，代替随机选取固定数量的图像进行对比的方法

非对比学习

□ 聚类方法—SwAV

• 方法

- 得到特征Z后，通过和C的最优传输算法得到对应的code
- Z和C之间进行相似度度量，得到的结果和code计算交叉熵损失



Caron M, Misra I, Mairal J, et al. Unsupervised learning of visual features by contrasting cluster assignments. NeurIPS 2020.

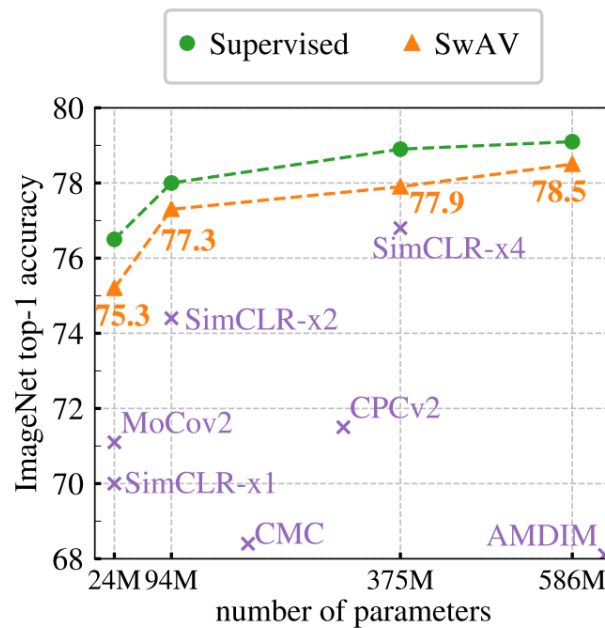
非对比学习

□ 聚类方法—SwAV

• 实验

➤ ImageNet-1K线性验证结果

Method	Arch.	Param.	Top1
Supervised	R50	24	76.5
Colorization [65]	R50	24	39.6
Jigsaw [46]	R50	24	45.7
NPID [58]	R50	24	54.0
BigBiGAN [15]	R50	24	56.6
LA [68]	R50	24	58.8
NPID++ [44]	R50	24	59.0
MoCo [24]	R50	24	60.6
SeLa [2]	R50	24	61.5
PIRL [44]	R50	24	63.6
CPC v2 [28]	R50	24	63.8
PCL [37]	R50	24	65.9
SimCLR [10]	R50	24	70.0
MoCov2 [11]	R50	24	71.1
SwAV	R50	24	75.3



Caron M, Misra I, Mairal J, et al. Unsupervised learning of visual features by contrasting cluster assignments. NeurIPS 2020.

K-Means作业

助教：赖伟

502023330042@smail.nju.edu.cn

2024年05月07日

作业要求

自选数据集（例如Iris数据集）实现K-means聚类算法，并进行验证集评估，**不能使用现成的算法库。**

注意：需自己动手实现核心代码，不能直接调用算法库！！

• **提交作业报告(.pdf)以及可运行代码文件(.py或.ipynb)，报告内容包括但不限于：**

- 数据的分析与处理
- K-means的设计原理和核心代码
- 实验结果评估(如总内聚度)
- 对实验结果的分析
- 对聚类结果的可视化结果。

• **扩展内容（加分项）：**

- 比较不同超参数对实验结果的影响
- 对于有标签验证集，如何进行聚类结果的评估
- 其他。

作业步骤(简单示例)

1. 核心逻辑

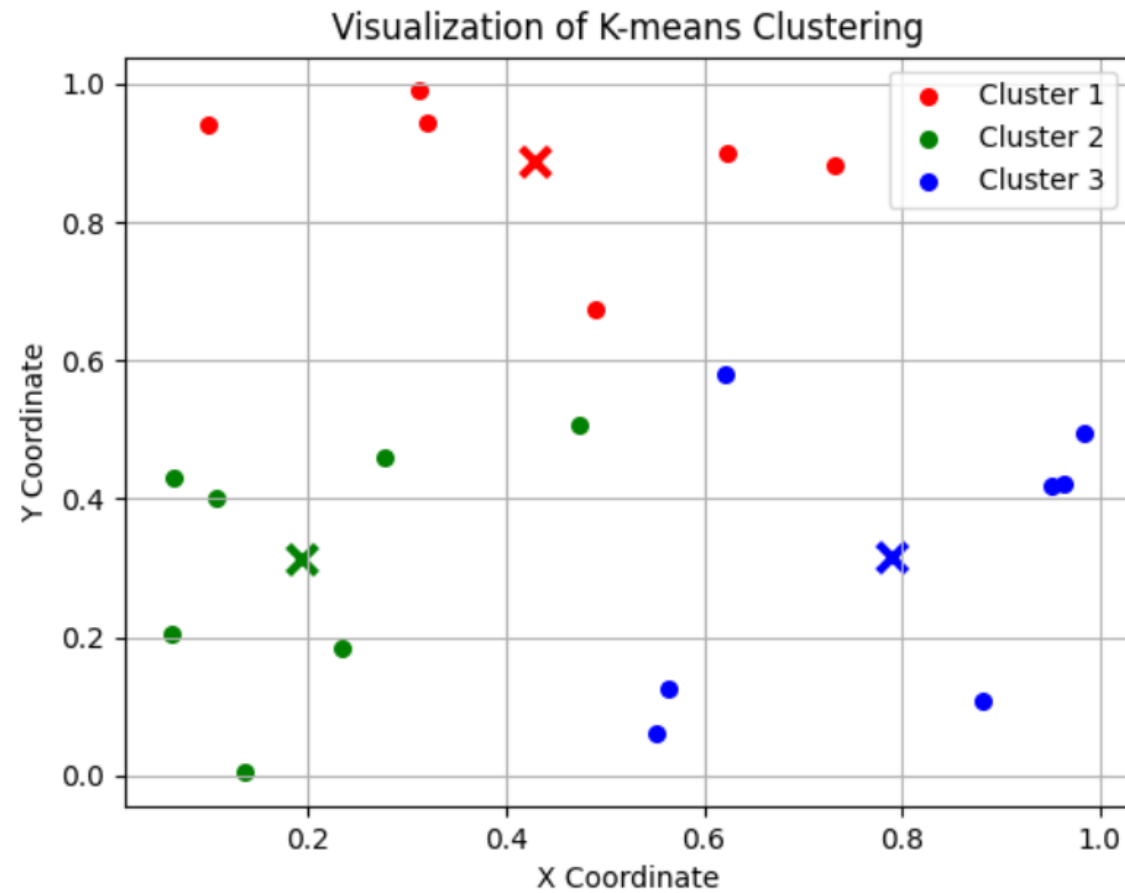
```
def k_means(points, k, max_iters=100):  
    centroids = initialize_centroids(points, k)  
    for _ in range(max_iters):  
        closest = closest_centroid(points, centroids)  
        centroids = move_centroids(points, closest, centroids)  
    return centroids, closest
```

2. 结果可视化

```
for i in range(k):  
    points_in_cluster = points[closest == i]  
    ax.scatter(points_in_cluster[:, 0], points_in_cluster[:, 1], s=30,  
c=colors[i], label=f'Cluster {i+1}')  
    ax.scatter(centroids[i][0], centroids[i][1], s=100, c=colors[i],  
marker='x', linewidths=3)
```

作业步骤(简单示例)

2. 可视化聚类结果(续)



作业提交

作业发送邮箱: nju_ml@163.com

主题命名: 2024_学号_神经网络_姓名 (附件同名)

截止时间: 5月27号 23:59

提交要求: 需要提交报告 (.pdf)和相关的代码文件，不要包含数据集，代码文件和报告压缩为一个压缩包作为邮件附件提交

参考资料

- K-means资料

<https://zhuanlan.zhihu.com/p/78798251>

<https://zhuanlan.zhihu.com/p/357072839>

谢谢!

联系方式: liwenbin@nju.edu.cn

更多信息: <https://cs.nju.edu.cn/liwenbin>